



Instituto Tecnológico
de Buenos Aires

mPES: Esquemas para Toma de Decisión Artificial en Escenarios Secuenciales

*Una evaluación comparativa de arquitecturas de
Aprendizaje por Refuerzo para la asignación de recursos limitados
bajo incertidumbre utilizando como escenario de prueba una crisis
pandémica*

Alumno: Ing. Maximiliano Leonel Vega

Director: Dr. Ing. Rodrigo Ramele

Tesis de Maestría

Maestría en Ciencia de Datos

Instituto Tecnológico de Buenos Aires

2026

Resumen

Esta tesis compara métodos de aprendizaje por refuerzo en un problema de asignación secuencial con recursos limitados. El escenario de prueba es una pandemia simulada: las ciudades se presentan sucesivamente, cada una con una severidad inicial, y un agente decide cuántos recursos asignarle de un presupuesto fijo. La severidad no atendida crece en cada paso, por lo que una asignación insuficiente se amplifica.

Se entrenaron seis agentes con hiperparámetros optimizados: dos tabulares (Q-Learning y Double Q-Learning) y cuatro redes neuronales (una red Q profunda, una variante recurrente que procesa los últimos estados, un método actor-crítico y un Transformer causal que atiende a la historia reciente), más un Q-Learning sin optimizar como referencia. Se evaluaron además seis reglas para combinar las redes en un ensamble, en las que cada red se pondera según su confianza en la decisión, derivada de la entropía de Shannon. Todos los modelos se evaluaron sobre las mismas 64 secuencias de ciudades en las condiciones de entrenamiento y en 21 escenarios de generalización, tres de ellos por encima de las cotas del entorno.

La métrica de comparación es el desempeño normalizado en cada secuencia, donde 0 representa la ausencia de asignación y 1 la asignación óptima, calculada con conocimiento de la secuencia completa. El Transformer es el mejor modelo individual, tanto en las condiciones de entrenamiento como en los escenarios de generalización.

Entre los ensambles, sólo el ensamble ponderado supera al Transformer individual, por un margen reducido. El que incluye una compuerta que regula la participación del Transformer alcanza un desempeño equivalente, porque ejecuta la acción de este en casi todas las decisiones. Esa ventaja se explica por su cercanía a una regla fija que asigna según la severidad y que, calibrada con la asignación óptima, alcanza el mismo nivel. Los ensambles que sólo promedian o votan con pesos derivados de la entropía de Shannon quedan por debajo del Transformer individual, por lo que esa confianza no basta por sí sola para superar al mejor miembro en desempeño promedio.

Palabras clave: aprendizaje por refuerzo, asignación secuencial con recursos limitados, Transformer causal, ensambles de modelos, entropía de Shannon, generalización.

Abstract

This thesis compares reinforcement learning methods on a sequential allocation problem with limited resources. A simulated pandemic is used as the test scenario: cities are presented sequentially, each with an initial severity, and an agent decides how many resources to allocate to each one from a fixed budget. Unaddressed severity grows at every step, so an insufficient allocation is amplified.

Six agents with optimised hyperparameters were trained: two tabular methods (Q-Learning and Double Q-Learning) and four neural networks (a deep Q-network, a recurrent variant that processes the most recent states, an actor-critic method and a causal Transformer that attends to the recent history), plus a non-optimised Q-Learning baseline. Six rules for combining the networks into an ensemble were also evaluated; in all of them each network is weighted by its confidence in the decision, derived from Shannon entropy. All models were evaluated on the same 64 sequences of cities under the training conditions and in 21 generalisation scenarios, three of them beyond the environment's bounds.

The comparison metric is the normalised performance on each sequence, where 0 represents no allocation and 1 the optimal allocation, computed with knowledge of the full sequence. The Transformer is the best individual model, both under training conditions and in the generalisation scenarios.

Among the ensembles, only the weighted ensemble outperforms the individual Transformer, by a small margin. The one that includes a gate regulating the participation of the Transformer achieves equivalent performance, since it executes the Transformer's action in almost every decision. This advantage is explained by its closeness to a fixed rule that allocates according to severity and that, calibrated against the optimal allocation, reaches the same level. Ensembles that only average or vote with weights derived from Shannon entropy fall below the individual Transformer, so this confidence is not sufficient on its own to outperform the best member in average performance.

Keywords: reinforcement learning, sequential allocation with limited resources, causal Transformer, model ensembles, Shannon entropy, generalisation.

Índice

1. Introducción	1
1.1. Motivación	1
1.2. Definición del problema	1
1.3. Preguntas e hipótesis	1
1.4. Contribuciones	2
2. Marco Teórico	3
2.1. Procesos de decisión de Markov	3
2.2. Aprendizaje por refuerzo basado en valores	4
2.3. Memoria, atención y actor-crítico	4
2.4. Entropía como medida de confianza	6
2.5. Ensamblés	6
2.6. Optimización bayesiana de hiperparámetros	7
2.7. Comparación estadística	7
3. Estado de la Cuestión	8
3.1. Aprendizaje por refuerzo para decisiones secuenciales bajo incertidumbre	8
3.2. Transformers en aprendizaje por refuerzo	9
3.3. Ensamblés en aprendizaje por refuerzo	9
3.4. Generalización bajo variaciones controladas	10
4. Materiales y Métodos	11
4.1. Implementación	11
4.2. Entorno	11
4.3. Métrica de desempeño	12
4.4. Modelos individuales	14
4.4.1. Modelos tabulares	14
4.4.2. Modelos con redes neuronales	14
4.5. Ensamblés	16
4.6. Optimización de hiperparámetros	18
4.7. Escenarios de generalización	19
4.8. Protocolo de evaluación	20
5. Resultados	21
5.1. Mejor modelo individual	21
5.2. Ensamblés	23
5.2.1. Efecto de las reglas fijas sobre la decisión	25
5.3. Transformer frente al mejor ensamble	26
5.4. Réplicas <i>held-out</i>	28
5.5. Estudio <i>post hoc</i> del ensamble ponderado	29
6. Discusión	33
6.1. Interpretación de la ventaja del Transformer	33
6.2. Efecto de la regla de combinación	33
6.3. Alcance de los resultados frente a una regla fija	34

7. Conclusiones	36
7.1. Respuesta a las preguntas de investigación	36
7.2. Limitaciones del estudio	36
7.3. Trabajo futuro	37
7.4. Conclusión general	37
Referencias Bibliográficas	38
Apéndice	43
A. Comandos de reproducción	43
B. Ejecución de la evaluación	43
C. Mapas estadísticos complementarios	44
D. Resultados por secuencia de cada modelo	48

Índice de figuras

4.1. Agente aleatorio	13
5.1. Desempeño medio por escenario (modelos individuales)	22
5.2. Curvas por secuencia en seis escenarios (modelos individuales)	23
5.3. Desempeño medio por escenario (ensambles)	26
5.4. Curvas de los ensambles en los escenarios fuera de rango	27
5.5. Resultados por secuencia: ensamble ponderado	28
C.1. d de Cohen entre pares (modelos individuales)	44
C.2. Welch por escenario (modelos individuales)	45
C.3. Welch por escenario (ensambles)	46
C.4. Welch entre pares (ensambles)	46
C.5. d de Cohen por escenario (ensambles)	47
C.6. d de Cohen entre pares (ensambles)	47
D.1. Resultados por secuencia: Q-Learning base	48
D.2. Resultados por secuencia: Q-Learning	48
D.3. Resultados por secuencia: Double Q-Learning	49
D.4. Resultados por secuencia: DQN	49
D.5. Resultados por secuencia: DQN recurrente	49
D.6. Resultados por secuencia: A2C	50
D.7. Resultados por secuencia: Transformer	50

Índice de tablas

4.1. Modelos individuales	14
4.2. Hiperparámetros de los modelos tabulares	14
4.3. Hiperparámetros de los modelos de tipo DQN	16
4.4. Hiperparámetros de A2C	16
4.5. Parámetros de los ensambles	19
4.6. Escenarios de evaluación	20

5.1. Desempeño de los modelos individuales	21
5.2. Desempeño de los ensambles	24
5.3. Comparación de cada ensamble con el Transformer	24
5.4. Frecuencia con que las reglas cambian la decisión	25
5.5. Transformer frente al ensamble ponderado	27
5.6. Referencia frente a las réplicas <i>held-out</i>	29
5.7. Sensibilidad del ensamble ponderado	30
5.8. Regla fija y modelos frente al óptimo	32
A.1. Comandos de reproducción	43

1. Introducción

Esta tesis compara arquitecturas de aprendizaje por refuerzo (RL, *Reinforcement Learning*) en una simulación de pandemia en la que un agente distribuye un presupuesto fijo entre ciudades que se presentan sucesivamente, sin conocer cuántas restan ni cuál será su severidad. El entorno, **mPES**, extiende el *Pandemic Experiment Scenario* (PES) del BCI-NE Lab de la Universidad de Essex [1].

1.1. Motivación

Durante la pandemia de COVID-19, gobiernos y hospitales debieron asignar recursos escasos cuando cada asignación modificaba la evolución posterior del sistema sanitario. En esta tesis la pandemia no es el objeto de estudio, sino un escenario controlado que permite medir qué modelos toman mejores decisiones y cómo se comportan fuera de las condiciones de entrenamiento.

1.2. Definición del problema

En cada paso se presenta una ciudad y el agente le asigna entre 0 y 10 unidades de recurso. La severidad de cada ciudad crece en un factor β por paso y cada unidad asignada la reduce en una cantidad α (Ecuación (4.3)). Esta regla simplifica los modelos de la epidemiología computacional [2], desde los compartimentales hasta simulaciones espaciales de COVID-19 basadas en agentes [3]: sólo conserva el crecimiento de la severidad y el efecto de los recursos.

1.3. Preguntas e hipótesis

1. ¿Qué modelo individual obtiene el mayor desempeño, y lo conserva cuando cambian la severidad o la longitud de las secuencias?
2. ¿Algún ensamble de modelos ya entrenados supera al mejor modelo individual y, en tal caso, con qué regla de combinación?

Para cada pregunta se plantea una hipótesis:

H1. *Entre los modelos individuales evaluados, el Transformer causal alcanza el mayor desempeño medio y lo conserva en los escenarios de generalización.*

H2. *Los ensambles de redes neuronales con votación basada en la confianza derivada de la entropía de Shannon superan el desempeño del mejor modelo individual.*

1.4. Contribuciones

1. Una comparación de seis modelos de RL, evaluados con el mismo protocolo y las mismas secuencias, con hiperparámetros de entrenamiento ajustados por optimización bayesiana.
2. Seis reglas para combinar las redes neuronales sin reentrenarlas, en las que cada red se pondera según la entropía de su distribución de acciones.
3. Una evaluación reproducible (Apéndice A) en 21 escenarios de generalización, 18 de los cuales cambian la severidad, la longitud de las secuencias o ambas.

2. Marco Teórico

Este capítulo resume los conceptos y métodos que usa la tesis; la referencia general es Sutton y Barto [4].

2.1. Procesos de decisión de Markov

Un proceso de decisión de Markov (MDP) finito es una tupla $\mathcal{M} = (\mathcal{S}, \mathcal{A}, P, r, \gamma)$, con un conjunto finito de estados $\mathcal{S}$, un conjunto finito de acciones $\mathcal{A}$, probabilidades de transición $P(s' | s, a)$, una función de recompensa r y un factor de descuento $\gamma \in [0, 1)$. Su rasgo definitorio es la propiedad de Markov: el estado siguiente depende sólo del estado y de la acción actuales.

$$P(s_{t+1} | s_t, a_t, s_{t-1}, a_{t-1}, \dots, s_0) = P(s_{t+1} | s_t, a_t) \quad (2.1)$$

Donde s_t y a_t son el estado y la acción en el paso t . El agente busca la política $\pi(a | s)$ que maximiza el retorno descontado esperado.

$$G_t = \sum_{k=0}^{T-t-1} \gamma^k r_{t+k+1} \quad (2.2)$$

Donde r_{t+k+1} es la recompensa recibida $k + 1$ pasos después de t y T el último paso del episodio. El valor de una acción bajo la política π es $Q^\pi(s, a) = \mathbb{E}_\pi[G_t | s_t = s, a_t = a]$, y el de la política óptima satisface la ecuación de Bellman.

$$Q^*(s, a) = \mathbb{E} \left[r_{t+1} + \gamma \max_{a'} Q^*(s_{t+1}, a') \mid s_t = s, a_t = a \right] \quad (2.3)$$

Donde la esperanza se toma sobre el estado siguiente y la recompensa. La política óptima elige en cada estado $\arg \max_a Q^*(s, a)$. Si la dinámica y toda la secuencia se conocen de antemano, la Ecuación (2.3) se resuelve exactamente por programación dinámica [4]; ese óptimo con información completa define aquí la cota superior de la métrica (Sección 4.3).

Cuando el agente no observa todas las variables de las que dependen la recompensa y la evolución futura, el problema es un MDP parcialmente observable (POMDP) [5]: la

observación actual no resume la historia, y las observaciones pasadas pueden aportar información útil para decidir.

2.2. Aprendizaje por refuerzo basado en valores

Los modelos tabulares y de tipo DQN de esta tesis estiman Q^* y eligen la acción de mayor valor con los siguientes métodos:

- **Q-Learning**: estima Q^* en una tabla [6] con la actualización $Q(s, a) \leftarrow Q(s, a) + \alpha_{lr} [r + \gamma \max_{a'} Q(s', a') - Q(s, a)]$, con tasa de aprendizaje α_{lr} . Explora con una política ε -greedy, que elige una acción al azar con probabilidad ε y la de mayor Q en otro caso; ε decrece de ε_0 a $\varepsilon_{\min}$.
- **Double Q-Learning**: mantiene dos tablas [7], una para elegir la acción siguiente y otra para evaluarla, con lo que reduce la sobreestimación que introduce el máximo.
- **Deep Q-Network (DQN)**: aproxima Q con una red neuronal [8] y estabiliza el entrenamiento con un *buffer* de repetición de experiencia (*experience replay*) [9] y una red objetivo (*target network*) que se copia cada C pasos. Double DQN [10] elige la acción siguiente con la red en línea y la evalúa con la red objetivo. Los tres agentes de tipo DQN de esta tesis usan ese valor objetivo y enmascaran las acciones no factibles. Se entrenan con Adam [11], con pesos iniciales de Glorot [12] y con dos mecanismos que acotan cada actualización: la pérdida de Huber [13] y el recorte de la norma del gradiente [14]. Como en [8], el aprendizaje comienza con un mínimo de transiciones en el *buffer*, de modo que los primeros lotes no provienen de pocos episodios.
- **Modelado de recompensa por potencial (PBRS)**: suma a la recompensa el término $\kappa[\gamma\Phi(s') - \Phi(s)]$ [15], que no cambia las políticas óptimas y puede acelerar el aprendizaje cuando la recompensa informa poco sobre el progreso; en esta tesis, $\Phi(s) = -\sum_i S_i$, con S_i las severidades actuales (Sección 4.2).

2.3. Memoria, atención y actor–crítico

En un POMDP puede procesarse una ventana de los últimos W estados, como hacen las dos primeras arquitecturas:

- **LSTM** [16]: recorre la ventana con un estado oculto y compuertas multiplicativas que regulan qué información se conserva. Hausknecht y Stone [17] mostraron que un DQN con una LSTM iguala al DQN con cuadros apilados en entornos parcialmente observables y se degrada menos cuando cambia la calidad de las observaciones.
- **Transformer** [18]: reemplaza la recurrencia por atención de producto escalar. Cada estado de la ventana se proyecta a dimensión d_{model} y se le suma un vector de posición que indica su orden; las W filas resultantes forman la matriz X . Una cabeza de atención combina las posiciones con pesos que dependen de su similitud.

$$\text{At}(X) = \text{softmax}\left(\frac{(XW_q)(XW_k)^\top}{\sqrt{d_h}} + M\right) XW_v \quad M_{ij} = \begin{cases} 0 & j \leq i \\ -\infty & j > i \end{cases} \quad (2.4)$$

Donde W_q , W_k y W_v son matrices aprendidas de $d_{\text{model}} \times d_h$, la *softmax* se aplica por filas y la máscara causal M impide que cada posición atienda a las posteriores. La atención de varias cabezas concatena h cabezas con parámetros propios y las proyecta de nuevo a d_{model} , $\text{MHA}(X) = [\text{At}_1(X), \dots, \text{At}_h(X)] W_o$. Cada bloque aplica esa atención y una capa *feed-forward* por posición, $\text{FF}(x) = \max(0, xW_1 + b_1) W_2 + b_2$, cada una con conexión residual y normalización de capa (LN).

$$X' = \text{LN}(X + \text{MHA}(X)) \quad X'' = \text{LN}(X' + \text{FF}(X')) \quad (2.5)$$

Donde X'' es la entrada del bloque siguiente; debido a la máscara, sólo la última fila depende de toda la ventana.

- **Actor-crítico**: estos métodos de gradiente de política [19], como A2C, aprenden a la vez una política (actor) y una función de valor (crítico). La ventaja se estima con GAE [20], que promedia estimaciones de varios pasos con un factor λ , y un término de entropía con coeficiente β_H evita la convergencia prematura de la política.

2.4. Entropía como medida de confianza

La entropía de Shannon [21] se define para una distribución p sobre las acciones.

$$H(p) = - \sum_{a \in \mathcal{A}} p(a) \log p(a) \quad (2.6)$$

Donde $p(a)$ es la probabilidad que el modelo asigna a la acción a . La entropía vale 0 si toda la probabilidad está en una acción y alcanza su máximo, $\log |\mathcal{A}|$, con la distribución uniforme. Es una medida habitual de la incertidumbre de un modelo [22]. En esta tesis se denomina *confianza* a uno menos la entropía normalizada por su máximo (Ecuación (4.8)), que va de 0, con la distribución uniforme, a 1. Los agentes de tipo DQN producen valores Q , no probabilidades, por lo que la confianza se calcula sobre una distribución derivada de ellos (Sección 4.5) y no es una probabilidad calibrada.

2.5. Ensambls

Un ensamble combina las salidas de varios modelos ya entrenados. Los valores Q de cada miembro se convierten en una distribución con una *softmax* de temperatura τ (Ecuación (4.7)): un τ pequeño concentra la probabilidad en la mejor acción y uno grande la acerca a la uniforme. En el voto suave (*soft voting*) se promedian las distribuciones con pesos w_k y se elige la acción más probable. En el voto duro (*hard voting*), aquí voto por acción, cada miembro vota por su acción preferida [23]. Los pesos pueden depender de la confianza de cada miembro en la decisión actual. Una compuerta, como en la mezcla de expertos [24], elige según la entrada qué miembro decide.

Las reglas de combinación pueden incorporar también conocimiento del dominio que no proviene de los miembros: mezclar la distribución del ensamble con una distribución previa (*prior*) sobre las acciones, o reemplazar las acciones que violan una restricción, como el blindaje (*shielding*) del RL seguro [25]. El *prior* de severidad y la cota de seguridad (Sección 4.5) son reglas de este tipo: asignan aproximadamente según la severidad y evitan asignar muy poco a ciudades de severidad alta.

2.6. Optimización bayesiana de hiperparámetros

Los hiperparámetros se ajustan con el estimador de Parzen estructurado en árbol (TPE) [26] de Optuna [27], que modela por separado la densidad de las configuraciones de mayor y de menor puntaje y propone configuraciones donde el cociente entre ambas es alto. Los ensayos poco prometedores se interrumpen con una regla de poda (*pruning*) que compara su puntaje intermedio con la mediana de los ensayos anteriores. En este trabajo se maximizó el desempeño promedio en las secuencias de referencia. Como la configuración se elige sobre esas mismas secuencias, su puntaje tiende a sobreestimar el desempeño en secuencias nuevas (sesgo de selección [28]). Por eso el desempeño se mide también en réplicas *held-out* de la misma distribución (Sección 5.4).

2.7. Comparación estadística

Las comparaciones entre modelos usan tres medidas:

- **Tamaño del efecto:** la d de Cohen [29] es la diferencia de medias expresada en desviaciones estándar y se interpreta como efecto despreciable ($|d| < 0,2$), pequeño (0,2 a 0,5), medio (0,5 a 0,8) o grande ($\geq 0,8$). Es la medida principal.
- **Significación:** la prueba t de Welch [30] indica si la diferencia es significativa, en los pares de modelos y en cada escenario frente a la referencia. El texto informa el orden de magnitud de p ($p < 10^{-k}$) y las figuras, $\log_{10} p$. Como cada par de modelos agrupa más de 1 300 secuencias por modelo, diferencias pequeñas producen valores p muy bajos: p indica si hay diferencia y d , cuán grande es. Además, todos los modelos se evalúan sobre las mismas secuencias y no se corrige por comparaciones múltiples, por lo que los valores p son orientativos.
- **Concordancia de ordenamientos:** la correlación de rangos de Spearman [31], r_s , correlaciona las posiciones de los modelos en dos ordenamientos; un valor cercano a 1 indica que el ordenamiento se conserva aunque cambien las medias.

3. Estado de la Cuestión

PES [1] se desarrolló para estudiar decisiones tomadas en conjunto por personas y agentes artificiales [32, 33]. Esta tesis utiliza únicamente el entorno; todas las decisiones son tomadas por agentes artificiales.

3.1. Aprendizaje por refuerzo para decisiones secuenciales bajo incertidumbre

Kochenderfer [34] y Powell [35] tratan el RL como una de varias familias de métodos para las decisiones secuenciales bajo incertidumbre, junto con la programación dinámica y la optimización estocástica.

Una primera línea aplica RL a la respuesta ante epidemias. Libin *et al.* [36] entrenaron un agente de gradiente de política (PPO, *Proximal Policy Optimization*) en un modelo metapoblacional de gripe pandémica con 379 distritos de Gran Bretaña. En un distrito contrastaron la política aprendida con una referencia conocida, que no pudieron establecer para el problema conjunto de 11 distritos. Ohi *et al.* [37] entrenaron un Double DQN con capas LSTM para fijar restricciones de movilidad en un escenario inspirado en COVID-19, equilibrando contagios y actividad económica. Bastani *et al.* [38] desplegaron en las fronteras de Grecia un sistema de RL que asignaba una cantidad limitada de pruebas e identificó 1,85 veces más viajeros infectados asintomáticos que el muestreo aleatorio, y Bednarski *et al.* [39] usaron RL para redistribuir equipamiento médico escaso entre estados de EE.UU.

Una segunda línea trata la asignación secuencial de recursos en salud y en operaciones. Komorowski *et al.* [40] aprendieron, a partir de registros de terapia intensiva, políticas de dosificación para pacientes con sepsis, y Yu *et al.* [41] revisan aplicaciones similares en salud. DeepRM [42] aprendió a asignar tareas con demandas de varios recursos con un desempeño comparable al de heurísticas establecidas; Gijbregts *et al.* [43] obtuvieron con A3C resultados equiparables a los de heurísticas y de programación dinámica aproximada en gestión de inventarios, y Kong *et al.* [44] mostraron que, en la mochila en línea (*online knapsack*), donde cada ítem se acepta o se rechaza al llegar sin conocer los siguientes,

agentes de RL aprenden comportamientos coherentes con los algoritmos óptimos conocidos. Dulac-Arnold *et al.* [45] formalizan los obstáculos para llevar estos métodos a sistemas reales.

mPES comparte con estos trabajos la asignación secuencial de un recurso escaso sin conocer la demanda futura, en una forma cercana a la mochila en línea.

Se diferencia en que el entorno es controlado y de bajo costo, el presupuesto es fijo por secuencia y existe un óptimo exacto con información completa, calculado por programación dinámica, contra el cual se normaliza el desempeño de cada secuencia. Además, en lugar de ajustar una política para un sistema real, esta tesis compara arquitecturas y ensambles sobre los mismos escenarios.

3.2. Transformers en aprendizaje por refuerzo

En RL, el Transformer se ha empleado de dos maneras. El *Decision Transformer* [46] aprende de trayectorias ya registradas y genera acciones condicionadas al retorno deseado. La segunda lo usa como codificador de la historia en un agente que aprende interactuando con el entorno: Parisotto *et al.* [47] mostraron que en ese caso el Transformer estándar es difícil de entrenar y que, con normalización de capa antes de cada subcapa y compuertas en las conexiones residuales, alcanza o supera el desempeño de las LSTM en tareas que requieren memoria. El Transformer de esta tesis sigue el segundo enfoque: es un DQN cuyo codificador procesa los últimos estados. Como el estado de mPES no incluye la severidad de las ciudades ya atendidas ni la longitud de la secuencia (Sección 4.2), comparar modelos con ventana (DQN recurrente y Transformer) y sin ella (DQN) indica si esa información mejora las decisiones.

3.3. Ensambls en aprendizaje por refuerzo

Lakshminarayanan *et al.* [22] mostraron que promediar redes de la misma arquitectura, entrenadas desde inicializaciones distintas, mejora la estimación de la incertidumbre. Wiering y van Hasselt [23] combinaron las políticas de cinco algoritmos distintos, entre ellos Q-Learning, Sarsa y actor-crítico, mediante votación y combinaciones de distribuciones de Boltzmann. Los ensambles de esta tesis siguen esa línea, pero combinan modelos ya

entrenados y ponderan a cada uno por la entropía de su distribución de acciones.

3.4. Generalización bajo variaciones controladas

Henderson *et al.* [48] encontraron que los algoritmos de RL profundo son sensibles a la semilla, a los hiperparámetros y a detalles de implementación; Cobbe *et al.* [49] mostraron que agentes entrenados con un número limitado de niveles obtienen un desempeño menor en niveles nuevos generados con la misma regla, y Kirk *et al.* [50] recomiendan evaluar sobre familias de variaciones controladas del entorno. Siguiendo esa recomendación, cada modelo se evalúa en 21 escenarios de generalización, tres de ellos por encima de las cotas del entorno (Sección 4.7).

4. Materiales y Métodos

4.1. Implementación

Los experimentos se implementaron en el repositorio `mPES` [51]. Cada paquete de `tabular`, `ml` y `ens` incluye el entorno (Gymnasium [52]), el entrenamiento y, cuando corresponde, la optimización bayesiana. Se utilizó Python 3.12 con TensorFlow 2.21, Keras 3.13, NumPy 2.4, Optuna 4.7, Gymnasium 1.2 y SciPy 1.17; los comandos de reproducción se encuentran en el Apéndice A.

4.2. Entorno

El entorno se formula como un MDP finito (Sección 2.1). En cada paso (*trial*) se presenta una ciudad y el agente decide cuántas unidades de un recurso escaso le asigna. Los estados y las acciones se definen como sigue.

$$\begin{aligned}\mathcal{S} &= \{(R, t, S) : R \in \{0, \dots, 30\}, t \in \{0, \dots, 10\}, S \in \{0, \dots, 9\}\} \\ \mathcal{A} &= \{0, 1, \dots, 10\}\end{aligned}\tag{4.1}$$

Donde R son los recursos restantes, t el paso dentro de la secuencia y S la severidad inicial de la ciudad actual; hay $31 \times 11 \times 10 = 3410$ estados. Cada secuencia dispone de 39 recursos, 9 de ellos preasignados por el entorno, por lo que el agente administra 30. Al elegir la acción a_t en el estado $s_t = (R_t, t, S_t)$, el entorno pasa al estado siguiente.

$$s_{t+1} = (R_t - \min(a_t, R_t), t + 1, S_{t+1})\tag{4.2}$$

Donde $\min(a_t, R_t)$ es la asignación efectiva (una asignación mayor que R_t se recorta) y S_{t+1} es la severidad inicial de la ciudad siguiente, que proviene de la secuencia y no depende de las acciones. La secuencia termina después de L pasos, con L desconocido para el agente. Como R y t evolucionan de forma determinista con la acción, la transición se modela como markoviana (Ecuación (2.1)). La severidad de toda ciudad i ya incorporada

evoluciona en cada paso.

$$S_{i,t+1} = \max(0, \beta S_{i,t} - \alpha a_i) \quad \alpha = 0,4 \quad \beta = 1 + \alpha = 1,4 \quad (4.3)$$

Donde a_i es la asignación que recibió la ciudad i al incorporarse, α la eficacia de cada unidad de recurso y β el factor de crecimiento por paso. Como la asignación se fija una sola vez y la severidad se actualiza en todos los pasos siguientes, el efecto de una asignación insuficiente se amplifica geométricamente. La recompensa de cada paso es la suma, con signo negativo, de las severidades de las ciudades activas.

$$r_{t+1} = - \sum_{i \leq t} S_{i,t+1} \quad (4.4)$$

Donde la suma recorre las ciudades incorporadas hasta el paso t . Con esta recompensa, maximizar el retorno (Ecuación (2.2)) equivale a minimizar la severidad acumulada. Las Ecuaciones (4.1)–(4.4) definen el MDP $\mathcal{M} = (\mathcal{S}, \mathcal{A}, P, r, \gamma)$; el descuento γ es un hiperparámetro de cada agente. Sin embargo, la recompensa depende de las severidades de las ciudades ya incorporadas y el final de la secuencia depende de L . Como ninguno forma parte de (R, t, S) , el problema es parcialmente observable para el agente (Sección 2.1). Los modelos con ventana de estados (Sección 4.4.2) procesan los últimos estados para recuperar parte de esa información.

Cada experimento consta de 8 bloques de 8 secuencias de 3 a 10 pasos (360 pasos en total). La referencia usa las 64 secuencias fijas de `initial_severity.csv` y `sequence_lengths.csv`, idénticas en todos los paquetes. En el entrenamiento se sortean longitudes y severidades de las distribuciones empíricas de esos mismos archivos.

4.3. Métrica de desempeño

Sea S_{agente} la suma de las severidades finales de las ciudades de una secuencia. Se normaliza entre dos cotas: S_{peor} , la que resulta de no asignar nada, y S_{mejor} , la mínima alcanzable con el presupuesto si se conoce de antemano toda la secuencia. Con ellas se define el desempeño normalizado.

$$\bar{r} = \frac{S_{\text{peor}} - S_{\text{agente}}}{S_{\text{peor}} - S_{\text{mejor}}} \quad (4.5)$$

Donde $\bar{r} = 0$ equivale a no asignar recursos y $\bar{r} = 1$ a la asignación óptima. Ambas cotas se calculan al terminar la secuencia y no intervienen en las decisiones.

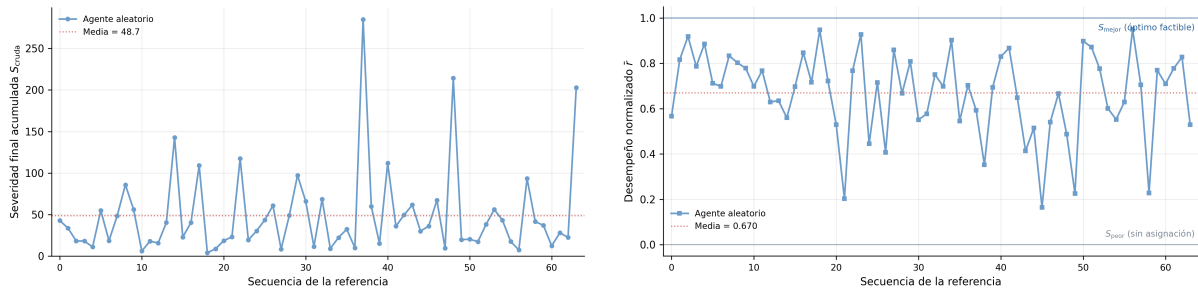
Asignación óptima. La asignación de cada ciudad se fija al incorporarse y su severidad no depende de las demás ciudades. Por eso, la severidad final de la ciudad c ($c = 0, \dots, L-1$) es una función $f_c(a)$ de su asignación a : el resultado de aplicar $L-c$ veces la Ecuación (4.3) a su severidad inicial. S_{mejor} minimiza $\sum_c f_c(a_c)$ con asignaciones enteras $0 \leq a_c \leq 10$ cuya suma no supera el presupuesto $B = 30$. Es un problema de mochila acotada que se resuelve en forma exacta por programación dinámica sobre las ciudades y los recursos restantes r .

$$V_c(r) = \min_{0 \leq a \leq \min(10, r)} [f_c(a) + V_{c+1}(r-a)] \quad V_L(r) = 0 \quad S_{\text{mejor}} = V_0(B) \quad (4.6)$$

Donde $V_c(r)$ es la mínima severidad final de las ciudades $c, \dots, L-1$ con r recursos disponibles. La recursión recorre las ciudades desde la última hasta la primera; guardando la acción que alcanza cada mínimo se reconstruye la asignación óptima de cada ciudad (Sección 5.5).

La Ecuación (4.6) es la forma finita y determinista de la ecuación de Bellman (Ecuación (2.3)), pero requiere conocer de antemano todas las severidades y la longitud L , que el agente no conoce: ningún agente puede superar ese óptimo ni garantizar alcanzarlo.

Un agente que elige acciones al azar obtiene $\bar{r} = 0,670$ en las 64 secuencias de la referencia (desviación estándar 0,186, rango [0,164; 0,950]; Figura 4.1). La mejor regla fija sin modelo, que asigna la severidad más dos unidades, $a = \min(S+2, R)$ (Sección 5.5), obtiene $\bar{r} = 0,940$ (desviación estándar 0,056, rango [0,744; 1,000]).



(a) Severidad final acumulada por secuencia.

(b) Desempeño normalizado $\bar{r}$ por secuencia.

Figura 4.1: Agente aleatorio en las 64 secuencias de la referencia; la línea punteada indica la media.

4.4. Modelos individuales

La Tabla 4.1 enumera los modelos, su paquete y el nombre que se usa en el texto; las figuras usan el nombre del paquete.

Tabla 4.1: Modelos individuales. Q-Learning base se incluye como referencia sin optimización.

Paquete	Nombre en el texto	Descripción
pes_base	Q-Learning base	Q-Learning tabular sin optimizar
pes_q1	Q-Learning	Q-Learning tabular optimizado
pes_dq1	Double Q-Learning	Double Q-Learning tabular con PBRS
pes_dqn	DQN	Double DQN con red densa
pes_rdqn	DQN recurrente	Double DQN con codificador LSTM
pes_a2c	A2C	Actor-crítico
pes_trf	Transformer	Double DQN con codificador Transformer causal

4.4.1. Modelos tabulares

Los modelos tabulares almacenan Q en una tabla de $31 \times 11 \times 10 \times 11 = 37\,510$ valores inicializados con $\mathcal{U}(-1, 1)$ y enmascaran las acciones que exceden los recursos. Una severidad mayor que 9 y un paso mayor que 10 se recortan a esas cotas. El modelo base usa los hiperparámetros originales de PES y Q-Learning, los de la optimización bayesiana; ambos reducen ε linealmente. Double Q-Learning mantiene ε_0 durante una fracción w de los episodios, la reduce exponencialmente hasta $\varepsilon_{\min}$ en la fracción q , usa PBRS (Sección 2.2) y decide con $\arg \max_a [Q_A(s, a) + Q_B(s, a)]$ (Tabla 4.2).

Tabla 4.2: Hiperparámetros de los modelos tabulares (Sección 2.2).

	Q-Learning base	Q-Learning	Double Q-Learning
Tasa de aprendizaje α_r	0,2	0,2862	0,1132
Descuento γ	0,9	0,8587	0,9777
$\varepsilon_0 / \varepsilon_{\min}$	0,8 / 0	0,6813 / 0,0437	0,5998 / 0,0327
Fracciones w / q	—	—	0,0260 / 0,6430
PBRS κ	—	—	0,000392
Episodios	1 000 000	550 000	360 000

4.4.2. Modelos con redes neuronales

Las redes reciben el estado escalado por las cotas del entorno, $[R/30, t/10, S/9]$. En los escenarios fuera de rango, las de tipo DQN reciben valores mayores que 1; A2C recorta S a 9. Todas usan Adam, inicialización Glorot uniforme y recorte de la norma del gradiente.

DQN, DQN recurrente y Transformer se entrenan como se describe en la Sección 2.2 (Double DQN, pérdida de Huber), con la exploración de Double Q-Learning y PBRS cuando $\kappa > 0$; el aprendizaje comienza cuando el *buffer* alcanza una fracción f de su capacidad. Difieren en el procesamiento de la entrada y terminan en una capa de 11 valores Q :

- **DQN**: dos capas densas de 64 unidades sobre el estado actual.
- **DQN recurrente** [17]: una LSTM de 64 unidades recorre los últimos $W = 6$ estados, completados con ceros al inicio de cada secuencia, y su último estado oculto pasa por dos capas densas de 96 unidades. El estado oculto no se conserva entre decisiones.
- **Transformer**: la misma ventana, proyectada a $d_{\text{model}} = 32$ y sumada a un vector de posición fijo, pasa por 2 bloques *Post-LN* (Ecuaciones (2.4) y (2.5)), sin las compuertas de Parisotto *et al.* [47], con atención causal de $h = 4$ cabezas de dimensión $d_h = 16$ y una capa *feed-forward* de 64 unidades, sin *dropout*. La última posición pasa por una capa densa de 32 unidades. La atención no enmascara las filas de ceros que completan la ventana: sólo aplica la máscara causal.

Las arquitecturas del DQN recurrente y del Transformer se eligieron mediante exploraciones *ad hoc*, porque incluirlas en la búsqueda bayesiana era demasiado costoso; la optimización sólo ajustó sus hiperparámetros de entrenamiento (Tabla 4.3).

A2C [53] (Sección 2.3) usa dos redes separadas con una capa oculta de 128 unidades: el actor, con salida *softmax* sobre las 11 acciones, y el crítico. La ventaja de GAE se estandariza por lote, y la política y su entropía se calculan sobre las acciones factibles (Tabla 4.4).

Tabla 4.3: Hiperparámetros de DQN, DQN recurrente y Transformer. Entre paréntesis, las transiciones que corresponden a la fracción f . Términos en la Sección 2.2.

	DQN	DQN recurrente	Transformer
Parámetros entrenables	5 131	34 027	27 019
Tasa de aprendizaje	0,001508	0,002415	0,000215
Descuento γ	0,9634	0,9719	0,9234
ε_0 / $\varepsilon_{\min}$	0,9627 / 0,0691	0,8883 / 0,0887	0,8651 / 0,0838
Fracciones w / q	0,2779 / 0,6290	0,1237 / 0,5990	0,2428 / 0,5333
Tamaño de lote	128	64	128
Capacidad del <i>buffer</i>	20 000	60 000	30 000
Sincronización C (pasos)	1 000	2 000	500
Recorte del gradiente	3,953	3,475	4,170
PBRs κ	0,0226	0,00263	0
Inicio del aprendizaje f	0,1615 (3 230)	0,2021 (12 128)	0,1217 (3 650)
Episodios	40 000	30 000	30 000

Tabla 4.4: Hiperparámetros de A2C (GAE, coeficiente de entropía y PBRs: Secciones 2.2 y 2.3).

Tasa de aprendizaje inicial (actor / crítico)	0,000648 / 0,004299
Decaimiento de la tasa	coseno, hasta el 23,75 % del valor inicial
Descuento γ / λ de GAE	0,8545 / 0,9135
Coeficiente de entropía β_H	0,00528
Recorte del gradiente (ambas redes)	1,200
PBRs κ	0,1527
Penalización de gasto	$r \leftarrow r - 0,01039 a$
Sesgo inicial del logit de $a = 10$	-1,389
Episodios	125 000

4.5. Ensambls

Los ensambles no se entrenan: cargan los modelos ya entrenados de DQN, DQN recurrente y Transformer, y cinco de las seis variantes incorporan el actor de A2C. Para cada miembro k , en un estado con R recursos, la distribución sobre las acciones factibles se obtiene de

sus valores Q con una *softmax* de temperatura τ (Sección 2.5).

$$p_k(a | s) = \frac{\exp(Q_k(s, a)/\tau)}{\sum_{a' \leq R} \exp(Q_k(s, a')/\tau)} \quad a \leq R \quad (4.7)$$

Donde $Q_k(s, a)$ es el valor que el miembro k asigna a la acción a ; en A2C, p_k es la salida del actor renormalizada sobre las acciones factibles. De p_k se obtienen la acción preferida del miembro, $a_k = \arg \max_a p_k(a | s)$, y su confianza.

$$c_k = 1 - \frac{H(p_k)}{\log |\mathcal{A}_R|} \quad (4.8)$$

Donde $H(p_k)$ es la entropía de Shannon de p_k (Ecuación (2.6)) y $|\mathcal{A}_R|$ el número de acciones factibles (el ensamble ponderado normaliza por $\log 11$). Con $R = 0$ la única acción factible es $a = 0$ y se ejecuta sin votar. El peso efectivo de cada miembro es $\tilde{w}_k = w_k c_k^\rho$, con w_k un peso base y $\rho \geq 0$ el exponente de la confianza. Algunas reglas usan también $\hat{Q}_k(a)$, los valores Q del miembro estandarizados sobre las acciones factibles (en A2C, sus probabilidades). Las seis reglas de combinación son:

- **Ensamble ponderado:** promedia las distribuciones de DQN, DQN recurrente y Transformer con pesos que crecen con la confianza.

$$p_{\text{ens}}(a | s) \propto \eta(a) \sum_{k=1}^3 w_k (0,1 + c_k) p_k(a | s) \quad \eta(a) = \begin{cases} 0,3 & a = 0, R > 0 \\ 1 & \text{en otro caso} \end{cases} \quad (4.9)$$

Donde w_k son los pesos base, normalizados a suma 1 (el Transformer recibe el 82%; Tabla 4.5), la *softmax* usa $\tau = 15$ y $\eta(a)$ reduce la probabilidad de no asignar nada mientras queden recursos. Esa distribución se mezcla con un *prior* centrado en la severidad actual.

$$p'(a | s) \propto (1 - w_\pi) p_{\text{ens}}(a | s) + w_\pi \pi_{\text{prior}}(a | s) \quad \pi_{\text{prior}}(a | s) \propto \exp \left[-\frac{(a - S)^2}{2\sigma_S^2} \right] \quad (4.10)$$

Donde S es la severidad de la ciudad actual, recortada a 9 (como en el estado que reciben los miembros), $\sigma_S = 3$ el ancho del *prior* y $w_\pi = 0,17$ su peso. Las dos distribuciones se normalizan sobre las acciones factibles. La acción ejecutada es la

de mayor p' , salvo que $S \geq 6$, esa acción sea menor que $\lfloor S/2 \rfloor$ y queden al menos $\lfloor S/2 \rfloor$ recursos: entonces una cota de seguridad la reemplaza por $\lfloor S/2 \rfloor$.

- **Voto suave:** elige la acción de mayor $\sum_k \tilde{w}_k p_k(a \mid s)$. Es la única variante optimizable que ajusta la temperatura; las otras cuatro usan $\tau = 1$.
- **Voto por acción:** cada miembro vota por a_k con peso $\tilde{w}_k$; los empates se resuelven con $\sum_k \hat{Q}_k$.
- **Consenso:** al voto ponderado le suma β_a por la confianza de los miembros que prefieren cada acción, le resta β_d por la de los que prefieren otra y agrega $\sum_k c_k \hat{Q}_k(a)$.
- **Consenso con *prior*:** conserva los términos β_a y β_d del consenso, pero reemplaza el voto y los $\hat{Q}_k$ por la suma $\sum_k \tilde{w}_k p_k$, atenúa la acción nula como η , mezcla el resultado con el *prior* de la Ecuación (4.10), con sus propios w_π y σ_S , y aplica la misma cota de seguridad.
- **Compuerta del Transformer:** calcula la compuerta logística $g = 1/\{1 + \exp[-\kappa_g(c_{\text{trf}} - \tau_g)]\}$, con c_{trf} la confianza del Transformer, umbral τ_g y pendiente κ_g ; si $g \geq 0,5$ (equivalente a $c_{\text{trf}} \geq \tau_g$, por lo que κ_g no afecta la decisión) ejecuta la acción del Transformer y, si no, la de un voto suave con todos los miembros.

Los parámetros del ensemble ponderado se fijaron *ad hoc*, observando su desempeño en las 64 secuencias de la referencia, y no provienen de una búsqueda sistemática; la Sección 5.5 los analiza *post hoc* mediante un estudio de sensibilidad y un contraste con la asignación óptima. En los otros cinco, los parámetros, incluido el peso de A2C, corresponden a los del mejor ensayo de Optuna sobre esas mismas secuencias (Tabla 4.5).

En esos cinco ensambles, el puntaje del estudio coincide con la media del ensemble en la referencia. Como las reglas no se entrenan junto con los miembros, las diferencias entre ensambles miden el efecto de la regla sobre los mismos modelos.

4.6. Optimización de hiperparámetros

Cada paquete optimizable ejecuta un estudio de Optuna con TPE [26, 27] que maximiza el $\bar{r}$ medio sobre las 64 secuencias de la referencia. En los modelos individuales, cada ensayo entrena el agente desde cero, el número de episodios forma parte de la búsqueda

Tabla 4.5: Parámetros de los ensambles: temperatura τ , exponente de confianza ρ y pesos base de DQN, DQN recurrente, Transformer y A2C.

Ensamble	τ	ρ	w_{DQN}	w_{rec}	w_{trf}	w_{A2C}	Otros
Ensamble ponderado	15	—	0,18	0,90	5,0	—	$w_{\pi} = 0,17$, $\sigma_S = 3$
Voto suave	1,219	2,681	0,270	1,760	0,896	0,0070	—
Voto por acción	1	2,197	0,468	2,599	1,803	0,174	—
Consenso	1	0,2754	1,532	1,931	2,504	1,881	$\beta_a = 2,074$, $\beta_d = 0,1211$
Consenso con <i>prior</i>	1	1,050	2,122	2,723	2,277	0,3366	$\beta_a = 0,2437$, $\beta_d = 0,1146$, $w_{\pi} = 0,1777$, $\sigma_S = 0,9097$
Compuerta del Transformer	1	1,452	0,837	2,561	2,601	0,0674	$\tau_g = 0,2222$, $\kappa_g = 2,397$

y MedianPruner interrumpe los ensayos poco prometedores; el modelo final se reentrena con la mejor configuración y la semilla $42 + i + 1$, con i el índice de ese ensayo.

4.7. Escenarios de generalización

Además de la referencia, cada modelo se evalúa en 21 escenarios de generalización (Tabla 4.6): 9 cambian la distribución de severidades, 5 las longitudes, 4 ambas y 3, de control, repiten las secuencias de la referencia, dos con otra partición en bloques. Ocho de los escenarios de severidad incluyen valores de 0, 1 o 9, que pertenecen al espacio de estados pero no aparecen en el entrenamiento. Tres escenarios quedan *fuera de rango*: la *extrapolación de severidad* (severidades de 10 a 12), las *secuencias largas* (11 a 20 pasos) y la *extrapolación conjunta* (ambas). Todos se generan con la semilla 42, por lo que los modelos enfrentan las mismas secuencias. La *degradación* en un escenario es $\bar{r}_{\text{ref}} - \bar{r}_{\text{esc}}$ (positiva si se pierde desempeño).

Como las optimizaciones eligen su configuración sobre las mismas 64 secuencias de la referencia, se agregan cinco réplicas *held-out* (`heldout_s1` a `heldout_s5`). Cada una consta de 8×8 secuencias nuevas, cuyas longitudes y severidades iniciales se sortean de forma independiente con las frecuencias observadas en la referencia, como en el entrenamiento. Las réplicas no intervienen en el entrenamiento, en la optimización ni en las medias de generalización. La diferencia entre la referencia y su media mide cuánto se sobreajustó cada configuración a esas 64 secuencias (Sección 5.4).

Tabla 4.6: Referencia, 21 escenarios de generalización y 5 réplicas *held-out*. Los que tienen **extrap** en el nombre superan las cotas del entorno ($S \leq 9$, $L \leq 10$).

Familia	Identificador	Descripción
referencia	sev_base	Distribución de entrenamiento.
severidad	sev_uniform	$U\{0, \dots, 9\}$.
severidad	sev_gauss_low	$\mathcal{N}(2, 1, 5)$ truncada, brotes leves.
severidad	sev_gauss_mid	$\mathcal{N}(4, 5, 2, 0)$ truncada, brotes medios.
severidad	sev_gauss_high	$\mathcal{N}(7, 1, 5)$ truncada, brotes intensos.
severidad	sev_weibull	Weibull($k = 1, 5$), cola superior pesada.
severidad	sev_beta_lowskew	$9 \cdot \text{Beta}(2, 5)$, sesgo bajo.
severidad	sev_beta_highskew	$9 \cdot \text{Beta}(5, 2)$, sesgo alto.
severidad	sev_bimodal	$0,5\mathcal{N}(2, 1) + 0,5\mathcal{N}(7, 1)$.
severidad	sev_extrapolate_high	$U\{10, 11, 12\}$, fuera de rango.
longitud	len_all_short	Todas las secuencias de longitud 3.
longitud	len_all_long	Todas las secuencias de longitud 10.
longitud	len_geometric	$2 + \text{Geom}(p = 0,2)$, recortada a $[3, 10]$.
longitud	len_poisson	$\text{Poisson}(\lambda = 5)$, recortada a $[3, 10]$.
longitud	len_extrapolate_long	$U\{11, \dots, 20\}$, fuera de rango.
conjunta	joint_high_long	Gaussiana alta $\times$ todas largas.
conjunta	joint_low_short	Gaussiana baja $\times$ todas cortas.
conjunta	joint_uniform_geom	Uniforme $\times$ geométrica.
conjunta	joint_extrap_both	Severidad y longitud fuera de rango.
estructural	struct_few_long_blocks	4 bloques $\times$ 16 secuencias.
estructural	struct_many_short_blocks	16 bloques $\times$ 4 secuencias.
estructural	struct_more_total	8 bloques $\times$ 8 secuencias, las de la referencia.
réplica	heldout_s1-s5	Sorteo independiente de longitudes y severidades con las frecuencias de la referencia (semilla $42 + k$).

4.8. Protocolo de evaluación

Cada modelo se evalúa en la referencia, en los 21 escenarios y en las 5 réplicas de forma determinista, con la acción factible de mayor valor (Apéndice B). Los modelos individuales, incluido Q-Learning base, y los ensambles forman dos grupos. Para cada modelo y escenario se calculan la media y la desviación estándar de $\bar{r}$ y la degradación. Las comparaciones entre dos modelos agrupan las secuencias de los 21 escenarios de generalización y se resumen con la d de Cohen y el valor p de Welch (Sección 2.7).

5. Resultados

Salvo indicación en contrario, los valores corresponden al desempeño normalizado $\bar{r}$ (Ecuación (4.5)) medio de cada escenario; *referencia* es la condición de entrenamiento y *generalización*, la media en los 21 escenarios. Las diferencias entre medias se expresan como variación porcentual respecto de la media de comparación. Los mapas estadísticos por escenario y las matrices de comparación entre pares se encuentran en el Apéndice C (Figuras C.1–C.6).

5.1. Mejor modelo individual

El Transformer es el mejor modelo individual (Tabla 5.1): tiene la mayor media en la referencia y en generalización, la menor dispersión entre secuencias y la mayor media en 15 de los 21 escenarios (Figura 5.1). En generalización supera al resto de los modelos optimizados entre un 3,4 % (DQN) y un 6,7 % (Q-Learning), con una ventaja de tamaño medio frente a DQN ($d = 0,50$), el DQN recurrente ($d = 0,57$) y Q-Learning ($d = 0,79$).

Tabla 5.1: Modelos individuales: media en la referencia, desviación estándar entre secuencias (σ), media en generalización, peor escenario y pérdida porcentual de ese escenario respecto de la referencia.

Modelo	Ref.	σ	Gen.	Peor escenario	Pérdida
Transformer	0,927	0,045	0,930	Secuencias largas (0,860)	7,3 %
DQN	0,894	0,055	0,899	Secuencias largas (0,841)	5,9 %
A2C	0,887	0,063	0,896	Secuencias largas (0,826)	6,9 %
DQN recurrente	0,899	0,049	0,889	Extrap. de severidad (0,831)	7,5 %
Double Q-Learning	0,896	0,048	0,877	Extrap. de severidad (0,785)	12,4 %
Q-Learning	0,887	0,061	0,871	Extrap. de severidad (0,762)	14,0 %
Q-Learning base	0,871	0,074	0,851	Extrap. de severidad (0,627)	28,0 %

Modelos individuales: desempeño normalizado medio por modelo y escenario

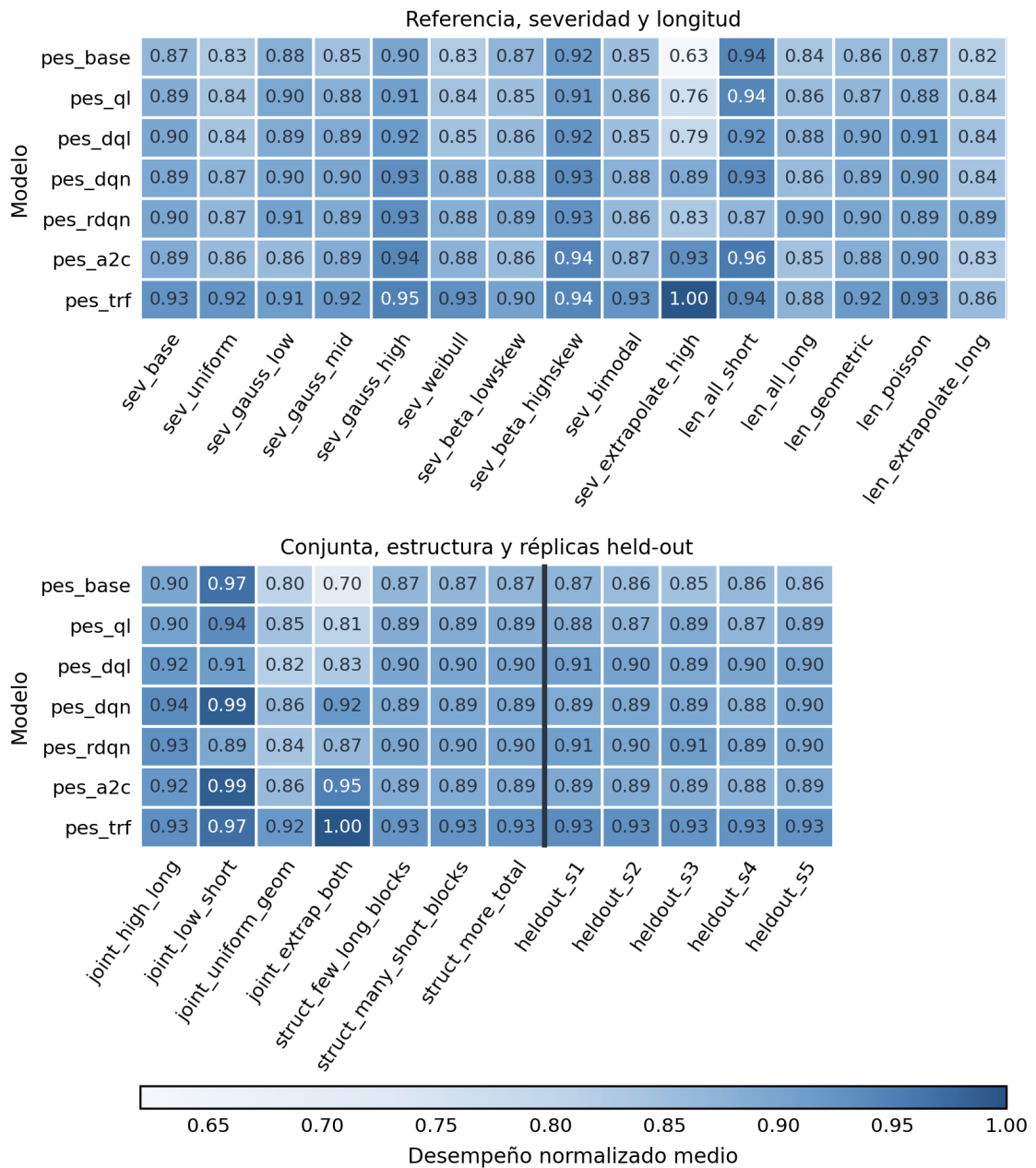


Figura 5.1: Modelos individuales: desempeño medio $\bar{r}$ por modelo y escenario. El panel superior muestra la referencia (primera columna), la severidad y la longitud; el inferior, los escenarios conjuntos y estructurales y, a la derecha de la línea vertical, las cinco réplicas *held-out*.

En la Figura 5.2, el Transformer queda por encima o al nivel del resto en los tres escenarios de severidad y en el de longitudes Poisson, con la mayor ventaja en los peores casos de la distribución bimodal; con secuencias largas lo supera el DQN recurrente, y con alta severidad y longitud 10 las curvas casi se superponen. Las Figuras D.1–D.7 (Apéndice D) muestran el desempeño por secuencia de cada modelo en la referencia.

**Modelos individuales: desempeño por secuencia y familia de perturbación
(trazo grueso = pes_trf)**

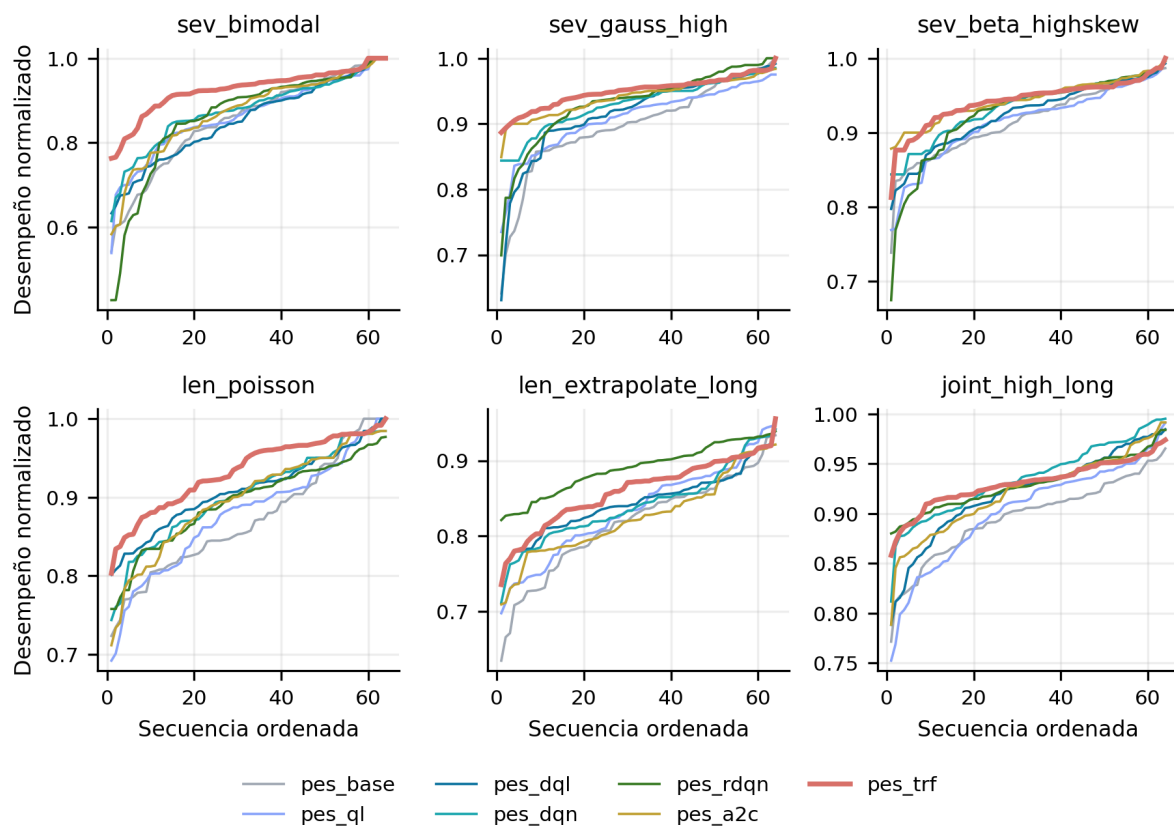


Figura 5.2: Desempeño por secuencia, ordenado de menor a mayor, en seis escenarios de generalización. Fila superior: severidad; fila inferior: longitud y conjunta. El trazo grueso corresponde al Transformer.

5.2. Ensamblas

La Tabla 5.2 resume los seis ensambles y la Tabla 5.3 compara cada uno con el Transformer individual; como los dos grupos se evaluaron por separado, d y p se calculan con la media, la desviación estándar y el número de secuencias de cada escenario.

El mejor ensamble es el ponderado: tiene la mayor media en la referencia y en generalización, el peor escenario más alto y la menor pérdida (Tabla 5.2), y es el único que supera al Transformer (Sección 5.3). Entre los ensambles, sólo lo superan el consenso, en `len_all_short` y `joint_low_short`, y la compuerta del Transformer, en `len_poisson` (Figura 5.3). Su ventaja es de tamaño medio sobre el consenso ($d = 0,58$) y despreciable sobre la compuerta ($d = 0,17$), ambas significativas ($p < 10^{-5}$). En la referencia reduce en un 45 % la distancia al óptimo que deja Q-Learning, $(\bar{r} - \bar{r}_{ql})/(1 - \bar{r}_{ql})$, frente al 36 %

del Transformer.

Tabla 5.2: Ensamblajes: media en la referencia, media en generalización, peor escenario y pérdida porcentual de ese escenario respecto de la referencia.

Ensamble	Ref.	Gen.	Peor escenario	Pérdida
Ensamble ponderado	0,937	0,939	Secuencias largas (0,900)	4,0 %
Compuerta del Transformer	0,928	0,931	Secuencias largas (0,861)	7,2 %
Consenso con <i>prior</i>	0,918	0,919	Secuencias largas (0,870)	5,2 %
Consenso	0,889	0,905	Secuencias largas (0,826)	7,1 %
Voto suave	0,914	0,902	Extrap. de severidad (0,860)	5,9 %
Voto por acción	0,914	0,901	Extrap. de severidad (0,859)	6,0 %

Tabla 5.3: Cada ensamble frente al Transformer individual en generalización. Diferencia: variación porcentual de la media del ensamble respecto de la del Transformer; los valores positivos indican ventaja del ensamble.

Ensamble	Diferencia	d	Efecto	p	Mayor media en
Ensamble ponderado	+1,1 %	0,19	despreciable	$< 10^{-5}$	18 de 21
Compuerta del Transformer	+0,1 %	0,02	despreciable	0,68	15 de 21
Consenso con <i>prior</i>	-1,1 %	-0,18	despreciable	$< 10^{-5}$	5 de 21
Consenso	-2,7 %	-0,39	pequeño	$< 10^{-22}$	3 de 21
Voto suave	-2,9 %	-0,40	pequeño	$< 10^{-24}$	2 de 21
Voto por acción	-3,1 %	-0,42	pequeño	$< 10^{-26}$	2 de 21

La compuerta del Transformer obtiene un desempeño equivalente al del Transformer ($d = 0,02$; $p = 0,68$): supera su media en 15 escenarios, pero nunca por más de un 0,6 %. Los otros cuatro ensambles quedan por debajo del Transformer individual. El voto suave y el voto por acción, que sólo promedian o votan ponderando por la confianza, tienen las medias más bajas (efecto pequeño, $d \approx -0,4$). Su peor escenario es la extrapolación de severidad: el 41 % y el 42 % de sus secuencias quedan por debajo de 0,8 (Figura 5.4) y su media es inferior a la de tres de sus cuatro miembros: en un 14 % a la del Transformer, entre un 7 % y un 8 % a la de A2C y en un 3 % a la de DQN.

5.2.1. Efecto de las reglas fijas sobre la decisi3n

Tres ensambles agregan reglas fijas al voto: el ensamble ponderado y el consenso con *prior* lo mezclan con un *prior* de severidad y aplican una cota de seguridad, y la compuerta del Transformer decide cu3ndo ejecutar la acci3n del Transformer. Para determinar si su decisi3n proviene del voto o de las reglas, se repiti3 su evaluaci3n registrando, en cada decisi3n, la acci3n antes y despu3s de cada regla y la que prefiere el Transformer (Ap3ndice B). La repetici3n reproduce las medias de la evaluaci3n principal con diferencias inferiores al 0,01 % de la escala de $\bar{r}$ (Tabla 5.4).

En el consenso con *prior* decide el voto: el *prior* cambia la acci3n votada en menos del 5 % de las decisiones. En la compuerta decide el Transformer: su acci3n se ejecuta en el 97 % y el 95 % de las decisiones, lo que explica la equivalencia de sus desempe3os.

En el ensamble ponderado deciden el voto y el *prior*: con $\tau = 15$ las distribuciones de los miembros quedan casi planas (en la referencia, la acci3n m3s probable de cada uno tiene en promedio probabilidad 0,19 y supera a la segunda en 0,02–0,03), y el *prior* cambia la acci3n votada en el 41 % y el 32 % de las decisiones; la cota, menos del 0,5 %.

En los dos ensambles que igualan o superan al Transformer, la decisi3n no proviene s3lo del voto ponderado por confianza, sino de la regla: el *prior* y la temperatura alta en el ensamble ponderado, y el umbral de confianza en la compuerta del Transformer.

Tabla 5.4: Porcentaje de las decisiones con recursos disponibles en que ocurre cada evento.

Ensamble	Evento	Ref.	Gen.
Compuerta del Transformer	Sigue al Transformer ($g \geq 0,5$)	96,9 %	94,9 %
	Acci3n final distinta de la del Transformer	0,8 %	2,7 %
Consenso con <i>prior</i>	El <i>prior</i> cambia la acci3n votada	4,5 %	4,6 %
	La cota cambia la acci3n	0,3 %	1,5 %
	Acci3n final distinta de la del Transformer	62,4 %	58,4 %
Ensamble ponderado	El <i>prior</i> cambia la acci3n votada	40,9 %	32,1 %
	La cota cambia la acci3n	0,0 %	0,3 %
	Acci3n final distinta de la del Transformer	61,2 %	47,4 %

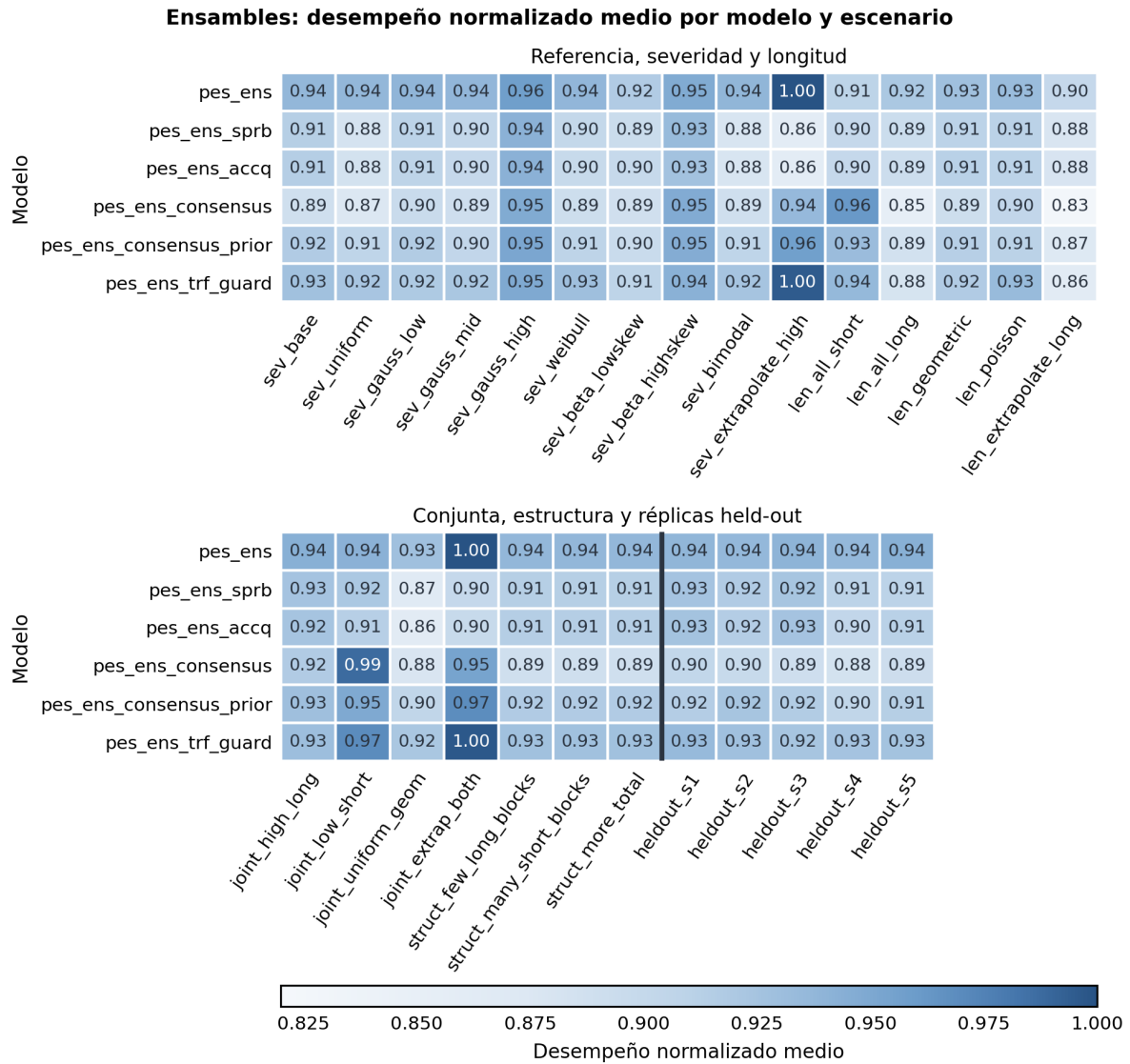


Figura 5.3: Ensamblas: desempeño medio $\bar{r}$ por ensemble y escenario, con la misma disposición que la Figura 5.1.

5.3. Transformer frente al mejor ensemble

El ensemble ponderado supera al Transformer en todas las medidas de la Tabla 5.5: su media en generalización es un 1,1% mayor, diferencia significativa pero de efecto despreciable ($d = 0,19$; Tabla 5.3); su peor escenario, un 4,7% más alto, y su mayor pérdida, poco más de la mitad. El Transformer sólo tiene mayor media en `len_all_short` y `joint_low_short` (un 3,0% y un 2,9% más) y, por menos del 0,1%, en `len_poisson`.

En la extrapolación de severidad y en la conjunta, el ensemble ponderado alcanza el óptimo en todas las secuencias ($\sigma = 0$; Figura 5.4), pero decide siempre como su miembro Transformer: la diferencia con el Transformer individual proviene de que el ensemble recorta la severidad a 9 antes de transmitirla a sus miembros.

La Figura 5.5 muestra su desempeño por secuencia en la referencia, donde tiene la menor dispersión de todos los modelos ($\sigma = 0,035$).

Tabla 5.5: Transformer individual frente al ensemble ponderado, el mejor ensemble.

Medida	Transformer	Ensemble ponderado
Media en la referencia (σ)	0,927 (0,045)	0,937 (0,035)
Media en generalización	0,930	0,939
Peor escenario (secuencias largas)	0,860	0,900
Mayor pérdida	7,3 %	4,0 %
Extrapolación de severidad	0,996	1,000
Extrapolación conjunta	0,997	1,000
Escenarios con mayor media (de 21)	3	18

Ensembles: desempeño en escenarios de extrapolación
(trazo grueso = pes_ens; línea punteada = media)

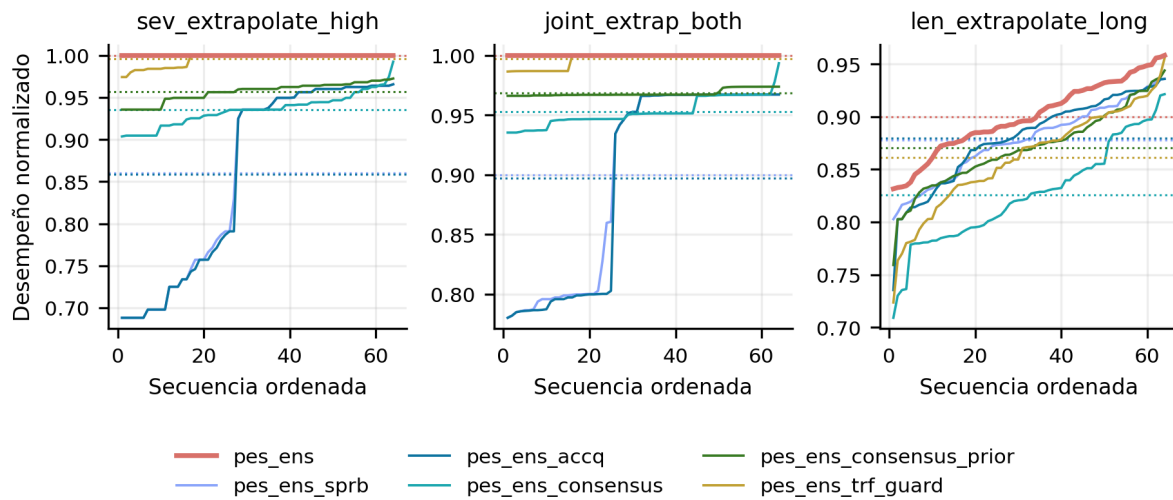


Figura 5.4: Desempeño por secuencia, ordenado de menor a mayor, de los seis ensambles en los escenarios fuera de rango; trazo grueso: ensemble ponderado; líneas punteadas: media de cada ensemble.

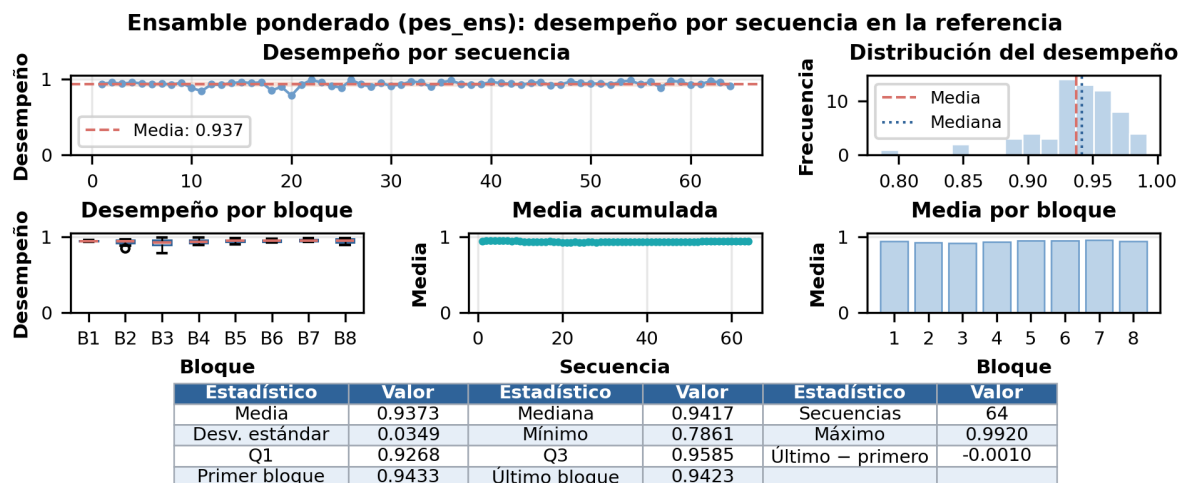


Figura 5.5: Resultados por secuencia del ensemble ponderado (*pes_ens*) en la referencia.

5.4. Réplicas *held-out*

En las cinco réplicas *held-out*, 320 secuencias nuevas de la distribución de la referencia, ningún modelo pierde desempeño de forma apreciable (Tabla 5.6). La caída va de $-0,5\%$ a $+1,2\%$, siempre con efecto despreciable ($|d| \leq 0,15$) y sin significación estadística ($p \geq 0,29$). En los once modelos optimizados con Optuna queda entre $-0,5\%$ y $+0,6\%$; la mayor caída corresponde a Q-Learning base, que no se optimizó, de modo que diferencias de ese tamaño son compatibles con la variación entre muestras. El ordenamiento se conserva (correlación de Spearman entre las medias, $r_s = 0,97$): sólo cambian de posición modelos cuyas medias difieren en a lo sumo un $0,5\%$ en ambas condiciones. En las mismas 320 secuencias, el ensemble ponderado supera al Transformer en un $1,1\%$ (prueba t pareada [54], $p < 10^{-7}$) y tiene mayor media en las cinco réplicas.

Elegir la configuración sobre las 64 secuencias de la referencia no produjo sobreajuste (*overfitting*): el sesgo de selección de la Sección 2.6 no sobrestimó el desempeño en la referencia, y cada modelo obtiene el mismo desempeño en secuencias que no intervinieron en su entrenamiento ni en su optimización.

Tabla 5.6: Media en la referencia y en las cinco réplicas *held-out* (entre paréntesis, desviación estándar de las medias de las réplicas); caída: variación porcentual de la referencia a las réplicas (positiva si el desempeño en las réplicas es menor); *d*: réplicas frente a referencia.

Modelo	Ref.	<i>Held-out</i>	Caída	<i>d</i>
Ensamble ponderado	0,937	0,939 (0,003)	−0,2 %	+0,04
Compuerta del Transformer	0,928	0,928 (0,004)	0,0 %	−0,01
Transformer	0,927	0,929 (0,003)	−0,2 %	+0,03
Consenso con <i>prior</i>	0,918	0,915 (0,007)	+0,3 %	−0,07
Voto por acción	0,914	0,916 (0,009)	−0,2 %	+0,04
Voto suave	0,914	0,917 (0,008)	−0,3 %	+0,06
DQN recurrente	0,899	0,903 (0,008)	−0,5 %	+0,08
Double Q-Learning	0,896	0,900 (0,006)	−0,4 %	+0,07
DQN	0,894	0,891 (0,006)	+0,3 %	−0,04
Consenso	0,889	0,893 (0,007)	−0,4 %	+0,06
A2C	0,887	0,887 (0,006)	0,0 %	−0,01
Q-Learning	0,887	0,881 (0,008)	+0,6 %	−0,08
Q-Learning base	0,871	0,860 (0,010)	+1,2 %	−0,15

5.5. Estudio *post hoc* del ensemble ponderado

Los parámetros del ensemble ponderado se fijaron *ad hoc* (Sección 4.5). Para evaluar esa elección se realizaron, después de la evaluación principal y sin modificar los parámetros, un análisis de sensibilidad y un contraste de cada regla del ensemble con la asignación óptima, ambos en la referencia y las cinco réplicas *held-out* (Apéndice B).

Sensibilidad. La Tabla 5.7 varía un parámetro por vez:

- **Temperatura, peso y ancho del *prior* y peso base del Transformer:** la media es máxima en el valor elegido, tanto en la referencia como en las réplicas, de modo que la elección no depende de las 64 secuencias sobre las que se realizó, pero se trata de máximos agudos: en las réplicas, un paso de la grilla reduce la media entre un 0,2 % y un 0,8 %.
- **Atenuación η , término 0,1 y cota de seguridad:** se encuentran en una meseta y modifican la media en a lo sumo un 0,1 %.

En las réplicas, sin *prior* ($w_\pi = 0$) el ensemble supera al Transformer en un 0,4 %; con el *prior* y $\tau = 1$, en un 0,1 %, y con ambos, en un 1,1 %. La ventaja proviene, por lo tanto, de combinar la temperatura alta con el *prior*.

Tabla 5.7: Sensibilidad del ensemble ponderado a cada parámetro, con los demás en sus valores fijos. La última columna indica los valores con los que supera al Transformer en las réplicas *held-out* por más de dos errores estándar pareados.

Parámetro	Fijo	Probados	Máximo	Supera al Transformer
Temperatura τ	15	1 a 50 (8)	15	2 a 30
Peso del <i>prior</i> w_π	0,17	0 a 0,5 (7)	0,17	0 a 0,25
Ancho del <i>prior</i> σ_S	3	1 a 5 (5)	3	3 a 5
Peso base del Transformer	5	1 a 10 (4)	5	2,5 a 10
Atenuación η	0,3	0,1 a 1 (4)	meseta	todos
Término $0,1 + c_k$	0,1	0 a 1 (4)	meseta	todos
Cota de seguridad	activa	sí / no	sin efecto	ambos

Contraste con el óptimo. La asignación óptima de cada ciudad se reconstruyó con la programación dinámica que calcula S_{mejor} (Ecuación (4.6)) en las 384 secuencias de la referencia y de las réplicas. En el 6 % de las ciudades, todas con $S \leq 5$, existe más de una acción óptima; los resultados no cambian al considerarlas todas.

- **Asignación óptima:** mientras queda presupuesto, el óptimo nunca asigna 0 y asigna en promedio 1,6 unidades más que la severidad de la ciudad (entre 0,7 y 1,9 según la severidad); cuando el presupuesto se agota, asigna 0.
- **Prior:** reproduce el ancho, pero no el centro. Una gaussiana centrada en S ajustada a esas asignaciones tiene $\sigma_S = 2,7$, cercano al 3 elegido; si también se ajusta el centro, este queda en $S + 2,2$, con $\sigma_S = 1,4$, y el ajuste mejora sustancialmente. El ancho del *prior* compensa en parte su centro desplazado.
- **Cota de seguridad:** nunca contradice al óptimo, que asigna al menos $\lfloor S/2 \rfloor$ siempre que puede, pero la acción del ensemble rara vez queda por debajo de ella: modifica 2 de 1 726 decisiones.
- **Atenuación η :** actúa en el sentido del óptimo, con un efecto despreciable.

- **Efecto de cada regla:** el *prior* es la única con efecto apreciable; cambia el 37 % de las decisiones con recursos y mejora el $\bar{r}$ medio en un 1,2 %. En las decisiones con recursos, el voto sin *prior* asigna en promedio $S + 2,2$ y el Transformer $S + 2,4$; con el *prior*, el ensamble asigna $S + 1,6$.

Una regla fija como política. Dado que el óptimo asigna algo más que la severidad, se evaluó también, sin ningún modelo, la regla $a = \min(S + k, R)$, que asigna la severidad de la ciudad más k unidades mientras alcancen los recursos. La dinámica (Ecuación (4.3)) explica por qué k debe ser positivo: con una asignación constante $a = S + k$, la severidad tras n pasos es

$$S_n = \max(0, S + k(1 - \beta^n)) \quad (5.1)$$

Con $k = 0$ la severidad permanece constante, porque la asignación sólo la contiene, y con cualquier $k > 0$ se extingue, tanto antes cuanto mayor es k : con $S = 7$, en 7, 5 y 4 pasos para $k = 1, 2$ y 3. El punto de equilibrio $a = S$ se deduce de la dinámica sin conocer el futuro; cuánto conviene superarlo depende de la holgura del presupuesto en las secuencias típicas.

La Tabla 5.8 evalúa la regla en las mismas secuencias que los modelos; como el óptimo con información completa vale $\bar{r} = 1$, la distancia de cada política al óptimo es $1 - \bar{r}$. Con $k = 2$, la regla queda a un 6 % del óptimo y por encima de doce de los trece modelos. Sólo el ensamble ponderado la iguala: la supera en un 0,2 % en generalización y queda un 0,4 % por debajo en las réplicas. En generalización, el Transformer queda un 0,8 % por debajo. Ningún modelo la supera en la referencia, y sólo el ensamble ponderado lo hace en una réplica. Los trece modelos la superan en los dos escenarios de secuencias cortas (`len_all_short`, 0,845, y `joint_low_short`, 0,866), donde la regla sigue asignando $S + 2$ y deja sin usar un excedente que los modelos aprendieron a aprovechar; con secuencias largas, en cambio, conviene un k menor.

El valor $k = 2$ se eligió a partir de la asignación óptima y es también el mejor en la referencia, sobre la que se ajustaron los modelos: la regla es un punto de comparación calibrado, no un competidor en igualdad de condiciones. Sin calibración ($k = 0$), queda muy por debajo de todos los modelos entrenados.

Tabla 5.8: Desempeño medio del óptimo con información completa, de la regla fija $a = \min(S + k, R)$, sin modelo, y de los trece modelos, en las mismas secuencias. Las dos últimas columnas comparan cada modelo con la regla $k = 2$ en los 21 escenarios de generalización: diferencia porcentual de medias y número de escenarios en que la supera.

Política	Ref.	Gen.	Réplicas	Dif. con $k = 2$	La supera
Óptimo (Ecuación (4.6))	1,000	1,000	1,000	—	—
Regla, $k = 0$ (asigna S)	0,782	0,805	0,787	—	—
Regla, $k = 1$	0,900	0,907	0,906	—	—
Regla, $k = 2$	0,940	0,937	0,943	—	—
Regla, $k = 3$	0,939	0,940	0,941	—	—
Ensamble ponderado	0,937	0,939	0,939	+0,2 %	4
Compuerta del Transformer	0,928	0,931	0,928	−0,7 %	3
Transformer	0,927	0,930	0,929	−0,8 %	3
Consenso con <i>prior</i>	0,918	0,919	0,915	−1,9 %	2
Consenso	0,889	0,905	0,893	−3,5 %	2
Voto suave	0,914	0,902	0,917	−3,7 %	2
Voto por acción	0,914	0,901	0,916	−3,9 %	2
DQN	0,894	0,899	0,891	−4,1 %	2
A2C	0,887	0,896	0,887	−4,4 %	2
DQN recurrente	0,899	0,889	0,903	−5,2 %	2
Double Q-Learning	0,896	0,877	0,900	−6,4 %	2
Q-Learning	0,887	0,871	0,881	−7,0 %	2
Q-Learning base	0,871	0,851	0,860	−9,3 %	2

6. Discusión

Los resultados respaldan la hipótesis H1 y no respaldan la H2: sólo una de las seis reglas, el ensamble ponderado, supera al Transformer, con una diferencia de tamaño despreciable (Sección 6.2). Sus parámetros, fijados *ad hoc*, se evaluaron *a posteriori* frente a la asignación óptima y a una regla fija calibrada con ella (Sección 5.5).

6.1. Interpretación de la ventaja del Transformer

La ventaja del Transformer no puede atribuirse a la ventana de estados: el DQN recurrente recibe la misma ventana de seis estados y en generalización queda un 4 % por debajo del Transformer y un 1 % por debajo del DQN, que sólo observa el estado actual. Tampoco se debe a la cantidad de parámetros: el Transformer tiene un 21 % menos que el DQN recurrente.

Descartados esos dos factores, la ventaja se atribuye a la atención causal, que pondera cada estado de la ventana según su relación con el estado actual en lugar de resumirla en un único estado oculto, como hace la LSTM. Como no se evaluó una variante sin atención, este efecto no puede separarse del de las conexiones residuales con normalización de capa ni del de los hiperparámetros.

La ventaja se conserva en generalización: el Transformer obtiene la mayor media en 15 de los 21 escenarios (Figura 5.1), con una media un 0,3 % mayor que la de la referencia. Su mayor pérdida se produce con secuencias más largas que las de entrenamiento, donde queda un 3 % por debajo del DQN recurrente.

6.2. Efecto de la regla de combinación

Los seis ensambles combinan los mismos modelos (el ponderado, sin A2C) y ponderan a cada miembro por su confianza; sólo difieren en la regla de combinación (Tabla 5.4):

- En el **ensamble ponderado**, el Transformer no determina la decisión pese a concentrar el 82 % del peso base: $\tau = 15$ aplana las distribuciones de los miembros y el *prior* cambia la acción votada en el 41 % de las decisiones de la referencia.

- La **compuerta del Transformer** lo sigue en al menos el 95 % de las decisiones, por lo que alcanza un desempeño equivalente.
- En el **voto suave** y el **voto por acción**, los pesos base favorecen al DQN recurrente sobre el Transformer; en la extrapolación de severidad, la ponderación por confianza no hace predominar al miembro de mayor desempeño individual.
- El **consenso con *prior*** mejora al consenso en la referencia y en su peor escenario; como su *prior* cambia menos del 5 % de las decisiones, la mejora no puede atribuirse a él.

La ventaja del ensamble ponderado proviene del *prior* y de la temperatura alta más que de la ponderación por confianza, y requiere ambos. Con las preferencias de los miembros casi planas, el *prior* acerca la decisión a la regla fija casi óptima (Sección 5.5). En las decisiones con recursos, reduce la asignación media de $S + 2,2$ (voto sin *prior*) a $S + 1,6$, lo que equivale a una regla con $k \approx 1,6$: el mismo exceso medio que la asignación óptima y 0,4 unidades menos que la regla $a = \min(S + 2, R)$. El Transformer, en cambio, asigna en promedio $S + 2,4$.

En las réplicas *held-out*, la temperatura alta sin *prior* y el *prior* con $\tau = 1$ superan al Transformer en a lo sumo un 0,4 %, frente al 1,1 % de ambos juntos (Sección 5.5). Los parámetros del ensamble ponderado están en máximos agudos, pero estos coinciden en la referencia y en las réplicas *held-out*, por lo que la ventaja se mantiene en ellas (Sección 5.4).

La ponderación por confianza, por sí sola, no basta para superar al mejor miembro: los dos ensambles que sólo promedian o votan con ella quedan por debajo del Transformer individual.

6.3. Alcance de los resultados frente a una regla fija

Una regla fija simple, que asigna la severidad más dos unidades, iguala al mejor ensamble y supera al Transformer (Tabla 5.8). Esto no invalida la comparación entre modelos: H1 y H2 comparan arquitecturas y reglas de combinación entre sí, en las mismas condiciones, y esas comparaciones se mantienen. Tampoco implica que los agentes no hayan aprendido. La dinámica sólo indica que conviene asignar más que la severidad (Ecuación (5.1)). El desplazamiento $k = 2$ se eligió a partir de la asignación óptima con información completa.

Sin esa calibración, la regla que asigna sólo la severidad obtiene 0,78 en la referencia, y los modelos individuales, entre 0,87 y 0,93 sólo a partir de la recompensa, sin conocer la regla ni el óptimo. Además, la regla no se adapta: el k conveniente varía con la longitud de las secuencias, y con secuencias cortas los trece modelos la superan porque aprendieron a utilizar el presupuesto que la regla deja sin asignar.

El resultado es análogo al de Kong *et al.* [44], en el que el RL redescubre los algoritmos clásicos de los problemas que estudia. Sin embargo, la regla fija requiere un óptimo calculable con información completa y expresable de forma simple, dos condiciones que provienen de las simplificaciones de mPES: ciudades que evolucionan de forma independiente, un único recurso y una dinámica determinista y conocida. Con ciudades que interactúan, varios recursos o una dinámica estocástica o desconocida, no existe un óptimo contra el cual calibrar una regla ni garantía de que la política óptima admita una expresión simple.

7. Conclusiones

7.1. Respuesta a las preguntas de investigación

Mejor modelo individual. Los resultados respaldan la hipótesis H1. El Transformer causal obtuvo el mayor desempeño medio entre los modelos individuales, en la referencia y en los escenarios de generalización. En estos últimos superó en un 3 % al DQN y en un 5 % al DQN recurrente, diferencias de tamaño medio. Mantuvo el desempeño con severidades fuera de rango, aunque no fue el más estable en su peor escenario: con secuencias largas perdió un 7 % respecto de su referencia, frente al 6 % del DQN.

Ensamblajes frente al mejor modelo individual. Los resultados no respaldan la hipótesis H2. Sólo el ensamble ponderado superó al Transformer, en un 1,1 % en generalización, una diferencia significativa pero de tamaño despreciable. La compuerta del Transformer alcanzó un desempeño equivalente y los otros cuatro ensambles quedaron por debajo. Su ventaja proviene de la temperatura alta y del *prior* de severidad más que de la ponderación por confianza: ambos acercan su decisión a la de la regla fija calibrada con el óptimo. El estudio *post hoc* muestra que la ventaja requiere ambos, que el *prior* está centrado por debajo de la asignación óptima y que la cota de seguridad no tiene efecto.

Ponderación por confianza. El diseño no permite determinar si ponderar por la confianza supera a una votación simple, porque todos los ensambles aplican esa ponderación. Por sí sola, no bastó para superar al Transformer.

7.2. Limitaciones del estudio

- Los hiperparámetros se ajustaron sobre la referencia; las réplicas *held-out* (Sección 5.4) no muestran sobreajuste, pero comparten la distribución de la referencia.
- Las arquitecturas del DQN recurrente y del Transformer se eligieron *ad hoc*; otras configuraciones podrían modificar el ordenamiento.
- Los parámetros del ensamble ponderado se fijaron *ad hoc* sobre la referencia y sólo se analizaron *post hoc*; están en máximos agudos de la sensibilidad.

- El entorno admite una regla fija casi óptima, por lo que los resultados no indican cuánto aportan estos modelos en problemas cuya política óptima no tenga una estructura simple.

7.3. Trabajo futuro

- Repetir la evaluación con semillas independientes para obtener intervalos de confianza del ordenamiento de la Tabla 5.1.
- Relacionar la confianza con el acierto de cada decisión mediante curvas de calibración.
- Aprender pesos $w_k(s)$ dependientes del estado.
- Enriquecer el entorno para que ninguna regla simple sea óptima, eliminando las simplificaciones que hacen simple a la política óptima (Sección 6.3): contagio entre ciudades; varios recursos con costos y eficacias distintos; una dinámica estocástica, o con α y β desconocidos y variables entre secuencias; y asignaciones que puedan repartirse en varios pasos en lugar de fijarse al incorporar cada ciudad.

7.4. Conclusión general

En mPES, el Transformer causal es el mejor modelo individual, y la combinación de modelos lo supera en un solo caso y por un margen despreciable: los resultados respaldan H1 y no respaldan H2. Una regla fija calibrada con la asignación óptima alcanza ese mismo nivel, porque la política óptima del entorno es simple.

Estas conclusiones se limitan al entorno y a los modelos evaluados. La regla requiere un óptimo calculable con información completa, que mPES admite por sus simplificaciones; los agentes evaluados, en cambio, aprenden sólo a partir de la recompensa. El marco desarrollado (entorno, protocolo, escenarios de generalización, réplicas *held-out* y reglas de combinación) se aplica sin cambios a variantes sin esas simplificaciones. En ese sentido, mPES actúa como un banco de pruebas en el que el óptimo conocido permite medir con exactitud cuánto se acercan los agentes, antes de aplicarlos a problemas en los que esa medición no es posible y en los que ninguna regla simple es casi óptima.

Referencias Bibliográficas

- [1] BCI-NE Lab, University of Essex. PES: Pandemic Experiment Scenario. <https://github.com/BCI-NE/PES>, 2023. Accedido: septiembre 2026.
- [2] Ellen Kuhl. *Computational Epidemiology: Data-Driven Modeling of COVID-19*. Springer, 2021. doi: 10.1007/978-3-030-82890-5.
- [3] Andrea Parisi, Samuel P. C. Brand, Joe Hilton, Rabia Aziza, Matt J. Keeling, and D. James Nokes. Spatially resolved simulations of the spread of COVID-19 in three European countries. *PLOS Computational Biology*, 17(7):e1009090, 2021. doi: 10.1371/journal.pcbi.1009090.
- [4] Richard S. Sutton and Andrew G. Barto. *Reinforcement Learning: An Introduction*. The MIT Press, Cambridge, MA, 2nd edition, 2018.
- [5] Leslie Pack Kaelbling, Michael L. Littman, and Anthony R. Cassandra. Planning and acting in partially observable stochastic domains. *Artificial Intelligence*, 101(1–2): 99–134, 1998. doi: 10.1016/S0004-3702(98)00023-X.
- [6] Christopher J. C. H. Watkins and Peter Dayan. Q-learning. *Machine Learning*, 8(3–4):279–292, 1992. doi: 10.1007/BF00992698.
- [7] Hado van Hasselt. Double Q-learning. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 23, pages 2613–2621, 2010.
- [8] Volodymyr Mnih, Koray Kavukcuoglu, David Silver, Andrei A. Rusu, Joel Veness, Marc G. Bellemare, Alex Graves, Martin Riedmiller, Andreas K. Fidjeland, Georg Ostrovski, et al. Human-level control through deep reinforcement learning. *Nature*, 518(7540):529–533, 2015. doi: 10.1038/nature14236.
- [9] Long-Ji Lin. Self-improving reactive agents based on reinforcement learning, planning and teaching. *Machine Learning*, 8(3–4):293–321, 1992. doi: 10.1007/BF00992699.
- [10] Hado van Hasselt, Arthur Guez, and David Silver. Deep reinforcement learning with double Q-learning. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 30, pages 2094–2100, 2016. doi: 10.1609/aaai.v30i1.10295.
- [11] Diederik P. Kingma and Jimmy Ba. Adam: A method for stochastic optimization. In *International Conference on Learning Representations (ICLR)*, 2015.
- [12] Xavier Glorot and Yoshua Bengio. Understanding the difficulty of training deep feedforward neural networks. In *Proceedings of the Thirteenth International Conference on Artificial Intelligence and Statistics (AISTATS)*, volume 9, pages 249–256, 2010.
- [13] Peter J. Huber. Robust estimation of a location parameter. *The Annals of Mathematical Statistics*, 35(1):73–101, 1964. doi: 10.1214/aoms/1177703732.
- [14] Razvan Pascanu, Tomas Mikolov, and Yoshua Bengio. On the difficulty of training recurrent neural networks. In *International Conference on Machine Learning (ICML)*, volume 28 of *Proceedings of Machine Learning Research*, pages 1310–1318, 2013.
- [15] Andrew Y. Ng, Daishi Harada, and Stuart Russell. Policy invariance under reward

- transformations: Theory and application to reward shaping. In *International Conference on Machine Learning (ICML)*, pages 278–287, 1999.
- [16] Sepp Hochreiter and Jürgen Schmidhuber. Long short-term memory. *Neural Computation*, 9(8):1735–1780, 1997. doi: 10.1162/neco.1997.9.8.1735.
- [17] Matthew Hausknecht and Peter Stone. Deep recurrent Q-learning for partially observable MDPs. In *AAAI Fall Symposium Series*, 2015.
- [18] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 30, 2017.
- [19] Ronald J. Williams. Simple statistical gradient-following algorithms for connectionist reinforcement learning. *Machine Learning*, 8(3–4):229–256, 1992. doi: 10.1007/BF00992696.
- [20] John Schulman, Philipp Moritz, Sergey Levine, Michael I. Jordan, and Pieter Abbeel. High-dimensional continuous control using generalized advantage estimation. In *International Conference on Learning Representations (ICLR)*, 2016.
- [21] Claude E. Shannon. A mathematical theory of communication. *The Bell System Technical Journal*, 27(3):379–423, 1948. doi: 10.1002/j.1538-7305.1948.tb01338.x.
- [22] Balaji Lakshminarayanan, Alexander Pritzel, and Charles Blundell. Simple and scalable predictive uncertainty estimation using deep ensembles. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 30, 2017.
- [23] Marco A. Wiering and Hado van Hasselt. Ensemble algorithms in reinforcement learning. *IEEE Transactions on Systems, Man, and Cybernetics, Part B (Cybernetics)*, 38(4):930–936, 2008. doi: 10.1109/TSMCB.2008.920231.
- [24] Robert A. Jacobs, Michael I. Jordan, Steven J. Nowlan, and Geoffrey E. Hinton. Adaptive mixtures of local experts. *Neural Computation*, 3(1):79–87, 1991. doi: 10.1162/neco.1991.3.1.79.
- [25] Mohammed Alshiekh, Roderick Bloem, Rüdiger Ehlers, Bettina Könighofer, Scott Niekum, and Ufuk Topcu. Safe reinforcement learning via shielding. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 32, 2018. doi: 10.1609/aaai.v32i1.11797.
- [26] James Bergstra, Rémi Bardenet, Yoshua Bengio, and Balázs Kégl. Algorithms for hyper-parameter optimization. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 24, pages 2546–2554, 2011.
- [27] Takuya Akiba, Shotaro Sano, Toshihiko Yanase, Takeru Ohta, and Masanori Koyama. Optuna: A next-generation hyperparameter optimization framework. In *ACM SIGKDD International Conference on Knowledge Discovery and Data Mining (KDD)*, pages 2623–2631, 2019. doi: 10.1145/3292500.3330701.
- [28] Gavin C. Cawley and Nicola L. C. Talbot. On over-fitting in model selection and subsequent selection bias in performance evaluation. *Journal of Machine Learning Research*, 11(70):2079–2107, 2010.

- [29] Jacob Cohen. *Statistical Power Analysis for the Behavioral Sciences*. Lawrence Erlbaum Associates, Hillsdale, NJ, 2nd edition, 1988.
- [30] Bernard L. Welch. The generalization of Student's problem when several different population variances are involved. *Biometrika*, 34(1/2):28–35, 1947.
- [31] Charles Spearman. The proof and measurement of association between two things. *The American Journal of Psychology*, 15(1):72–101, 1904. doi: 10.2307/1412159.
- [32] Akashdeep Nijjar, Ittipat Promnorakid, Riccardo Poli, Caterina Cinel, Thomas Reed, Christopher Baker, Stephen Hinton, and Stephen Fairclough. Enhancing decision-making in human-machine teams. In *2025 IEEE International Conference on Metrology for eXtended Reality, Artificial Intelligence and Neural Engineering (MetroXRaine)*, pages 13–18, 2025. doi: 10.1109/metroxraine66377.2025.11340006.
- [33] Tasos Papastyliaou, Rodrigo Ramele, Luca Citi, Caterina Cinel, and Riccardo Poli. PES – Pandemic Emergency Scenario. OpenNeuro, conjunto de datos ds004477, versión 1.0.2, 2023. URL <https://doi.org/10.18112/openneuro.ds004477.v1.0.2>.
- [34] Mykel J. Kochenderfer. *Decision Making Under Uncertainty: Theory and Application*. The MIT Press, Cambridge, MA, 2015. doi: 10.7551/mitpress/10187.001.0001.
- [35] Warren B. Powell. *Reinforcement Learning and Stochastic Optimization: A Unified Framework for Sequential Decisions*. Wiley, Hoboken, NJ, 2022. doi: 10.1002/9781119815068.
- [36] Pieter J. K. Libin, Arno Moonens, Timothy Verstraeten, Fabian Perez-Sanjines, Niel Hens, Philippe Lemey, and Ann Nowé. Deep reinforcement learning for large-scale epidemic control. In *Machine Learning and Knowledge Discovery in Databases. Applied Data Science and Demo Track (ECML PKDD 2020)*, volume 12461 of *Lecture Notes in Computer Science*, pages 155–170. Springer, 2021. doi: 10.1007/978-3-030-67670-4_10.
- [37] Abu Quwsar Ohi, M. F. Mridha, Muhammad Mostafa Monowar, and Md. Abdul Hamid. Exploring optimal control of epidemic spread using reinforcement learning. *Scientific Reports*, 10:22106, 2020. doi: 10.1038/s41598-020-79147-8.
- [38] Hamsa Bastani, Kimon Drakopoulos, Vishal Gupta, Ioannis Vlachogiannis, Christos Hadjichristodoulou, Pagona Lagiou, Gkikas Magiorkinis, Dimitrios Paraskevis, and Sotirios Tsiodras. Efficient and targeted COVID-19 border testing via reinforcement learning. *Nature*, 599(7883):108–113, 2021. doi: 10.1038/s41586-021-04014-z.
- [39] Bryan P. Bednarski, Akash Deep Singh, and William M. Jones. On collaborative reinforcement learning to optimize the redistribution of critical medical supplies throughout the COVID-19 pandemic. *Journal of the American Medical Informatics Association*, 28(4):874–878, 2021. doi: 10.1093/jamia/ocaa324.
- [40] Matthieu Komorowski, Leo A. Celi, Omar Badawi, Anthony C. Gordon, and A. Aldo Faisal. The Artificial Intelligence Clinician learns optimal treatment strategies for sepsis in intensive care. *Nature Medicine*, 24(11):1716–1720, 2018. doi: 10.1038/s41591-018-0213-5.
- [41] Chao Yu, Jiming Liu, Shamim Nemati, and Guosheng Yin. Reinforcement learning

- in healthcare: A survey. *ACM Computing Surveys*, 55(1):5:1–5:36, 2021. doi: 10.1145/3477600.
- [42] Hongzi Mao, Mohammad Alizadeh, Ishai Menache, and Srikanth Kandula. Resource management with deep reinforcement learning. In *Proceedings of the 15th ACM Workshop on Hot Topics in Networks (HotNets)*, pages 50–56, 2016. doi: 10.1145/3005745.3005750.
- [43] Joren Gijsbrechts, Robert N. Boute, Jan A. Van Mieghem, and Dennis J. Zhang. Can deep reinforcement learning improve inventory management? Performance on lost sales, dual-sourcing, and multi-echelon problems. *Manufacturing & Service Operations Management*, 24(3):1349–1368, 2022. doi: 10.1287/msom.2021.1064.
- [44] Weiwei Kong, Christopher Liaw, Aranyak Mehta, and D. Sivakumar. A new dog learns old tricks: RL finds classic optimization algorithms. In *International Conference on Learning Representations (ICLR)*, 2019. URL <https://openreview.net/forum?id=rkluJ2R9KQ>.
- [45] Gabriel Dulac-Arnold, Nir Levine, Daniel J. Mankowitz, Jerry Li, Cosmin Paduraru, Sven Gowal, and Todd Hester. Challenges of real-world reinforcement learning: definitions, benchmarks and analysis. *Machine Learning*, 110(9):2419–2468, 2021. doi: 10.1007/s10994-021-05961-4.
- [46] Lili Chen, Kevin Lu, Aravind Rajeswaran, Kimin Lee, Aditya Grover, Michael Laskin, Pieter Abbeel, Aravind Srinivas, and Igor Mordatch. Decision Transformer: Reinforcement learning via sequence modeling. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 34, pages 15084–15097, 2021.
- [47] Emilio Parisotto, H. Francis Song, Jack W. Rae, Razvan Pascanu, Caglar Gulcehre, Siddhant M. Jayakumar, Max Jaderberg, Raphaël Lopez Kaufman, Aidan Clark, Seb Noury, Matthew M. Botvinick, Nicolas Heess, and Raia Hadsell. Stabilizing transformers for reinforcement learning. In *International Conference on Machine Learning (ICML)*, volume 119 of *Proceedings of Machine Learning Research*, pages 7487–7498, 2020.
- [48] Peter Henderson, Riashat Islam, Philip Bachman, Joelle Pineau, Doina Precup, and David Meger. Deep reinforcement learning that matters. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 32, pages 3207–3214, 2018. doi: 10.1609/aaai.v32i1.11694.
- [49] Karl Cobbe, Christopher Hesse, Jacob Hilton, and John Schulman. Leveraging procedural generation to benchmark reinforcement learning. In *International Conference on Machine Learning (ICML)*, volume 119 of *Proceedings of Machine Learning Research*, pages 2048–2056, 2020.
- [50] Robert Kirk, Amy Zhang, Edward Grefenstette, and Tim Rocktäschel. A survey of zero-shot generalisation in deep reinforcement learning. *Journal of Artificial Intelligence Research*, 76:201–264, 2023. doi: 10.1613/jair.1.14174.
- [51] Maximiliano Vega. mPES: Multiple Pandemic Experiment Suite. <https://github.com/Maximiliano0/mPES>, 2026. Repositorio oficial. Accedido: septiembre 2026.
- [52] Mark Towers, Ariel Kwiatkowski, Jordan Terry, John U. Balis, Gianluca De Cola,

- Tristan Deleu, Manuel Goulão, Andreas Kallinteris, Markus Krimmel, Arjun KG, Rodrigo Perez-Vicente, Andrea Pierré, Sander Schulhoff, Jun Jet Tai, Hannah Tan, and Omar G. Younis. Gymnasium: A standard interface for reinforcement learning environments. In *Advances in Neural Information Processing Systems (NeurIPS), Datasets and Benchmarks Track*, 2025. URL <https://arxiv.org/abs/2407.17032>. arXiv:2407.17032.
- [53] Volodymyr Mnih, Adrià Puigdomènech Badia, Mehdi Mirza, Alex Graves, Timothy Lillicrap, Tim Harley, David Silver, and Koray Kavukcuoglu. Asynchronous methods for deep reinforcement learning. In *International Conference on Machine Learning (ICML)*, volume 48 of *Proceedings of Machine Learning Research*, pages 1928–1937, 2016.
- [54] Student. The probable error of a mean. *Biometrika*, 6(1):1–25, 1908. doi: 10.2307/2331554.

Apéndice

A. Comandos de reproducción

Todos los comandos se ejecutan desde el directorio `h1/` del repositorio `mPES`, con el entorno virtual `win_mpes_env` activado, Python 3.12, las dependencias de `utils/config/requirements.txt` y las variables `VIRTUAL_ENV`, `PYTHONIOENCODING=utf-8` y `TF_ENABLE_ONEDNN_OPTS=0` definidas (Tabla A.1). El entrenamiento final lee los hiperparámetros de `inputs/best_params.json` (Q-Learning base usa los valores originales de PES, fijados en su script). La arquitectura del DQN recurrente y del Transformer, y el número de episodios de las tres redes de tipo DQN, se leen de `config/CONFIG.py`.

Tabla A.1: Comandos de optimización bayesiana, entrenamiento final y evaluación de cada paquete. `<pkg>` designa cualquiera de los cinco ensambles optimizables.

Paquete	Optimización bayesiana	Entrenamiento final	Evaluación
<code>pes_base</code>	—	<code>python -m tabular.pes_base.ext.train_rl</code>	<code>python -m tabular.pes_base</code>
<code>pes_ql</code>	<code>python -m tabular.pes_ql.ext.optimize_rl</code>	<code>python -m tabular.pes_ql.ext.train_rl</code>	<code>python -m tabular.pes_ql</code>
<code>pes_dql</code>	<code>python -m tabular.pes_dql.ext.optimize_rl</code>	<code>python -m tabular.pes_dql.ext.train_rl</code>	<code>python -m tabular.pes_dql</code>
<code>pes_dqn</code>	<code>python -m ml.pes_dqn.ext.optimize_dqn</code>	<code>python -m ml.pes_dqn.ext.train_dqn</code>	<code>python -m ml.pes_dqn</code>
<code>pes_rdqn</code>	<code>python -m ml.pes_rdqn.ext.optimize_rdqn</code>	<code>python -m ml.pes_rdqn.ext.train_rdqn</code>	<code>python -m ml.pes_rdqn</code>
<code>pes_a2c</code>	<code>python -m ml.pes_a2c.ext.optimize_a2c</code>	<code>python -m ml.pes_a2c.ext.train_a2c</code>	<code>python -m ml.pes_a2c</code>
<code>pes_trf</code>	<code>python -m ml.pes_trf.ext.optimize_tr</code>	<code>python -m ml.pes_trf.ext.train_transformer</code>	<code>python -m ml.pes_trf</code>
<code>pes_ens</code>	—	—	<code>python -m ens.pes_ens</code>
<code><pkg></code>	<code>python -m ens.<pkg>.ext.optimize_ens</code>	—	<code>python -m ens.<pkg></code>

B. Ejecución de la evaluación

La evaluación en la referencia, los 21 escenarios y las cinco réplicas *held-out* se ejecuta desde `h1/`:

```
python -m general.scripts.benchmark run --suite both
python -m general.scripts.benchmark heldout
python -m general.scripts.analysis
python -m general.scripts.figures
python -m general.scripts.random_baseline
```

El primer comando, reanudable, evalúa cada par (modelo, escenario) de los dos grupos reemplazando temporalmente los archivos de entrada de cada paquete por los del

escenario. El segundo guarda en `general/results/heldout/` una copia de las réplicas y comprueba que los 13 modelos recibieron las mismas secuencias; cada réplica registra en `sampling_distribution.json` las frecuencias de sorteo, la semilla y las frecuencias obtenidas. El tercero calcula las matrices modelo $\times$ escenario y las comparaciones entre pares; el cuarto genera las figuras, y el quinto, las del agente aleatorio. Los resultados se guardan en `general/results/<grupo>/`.

Las frecuencias de la Tabla 5.4 se obtienen, desde la raíz del repositorio, con `python writings/auxiliar/scripts/ensemble_decisions.py`, que repite la evaluación del ensamble ponderado, del consenso con *prior* y de la compuerta del Transformer, registra en cada decisión la acción antes y después de cada regla y comprueba que las medias coincidan con las de la evaluación principal. La Tabla 5.3 se obtiene con `python writings/auxiliar/scripts/trf_vs_ens.py`, que compara cada ensamble con el Transformer a partir de los resultados guardados en `general/results/`, y la Tabla 5.6, con `python writings/auxiliar/scripts/heldout_gap.py`. Los análisis de la Sección 5.5 se obtienen del mismo modo con `weighted_ens_sensitivity.py` (sensibilidad), `weighted_ens_oracle.py` (asignación óptima y efecto de cada regla) y `fixed_rule.py` (regla fija en los 27 escenarios).

C. Mapas estadísticos complementarios

Las figuras de esta sección detallan, modelo por modelo, las comparaciones estadísticas que el texto presenta de forma agregada.

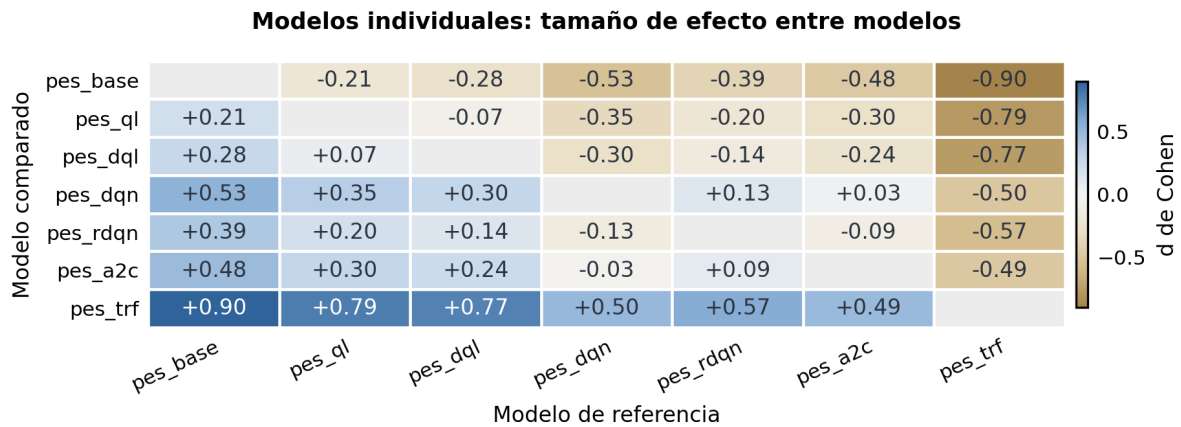


Figura C.1: Modelos individuales: d de Cohen entre pares de modelos en los 21 escenarios de generalización.

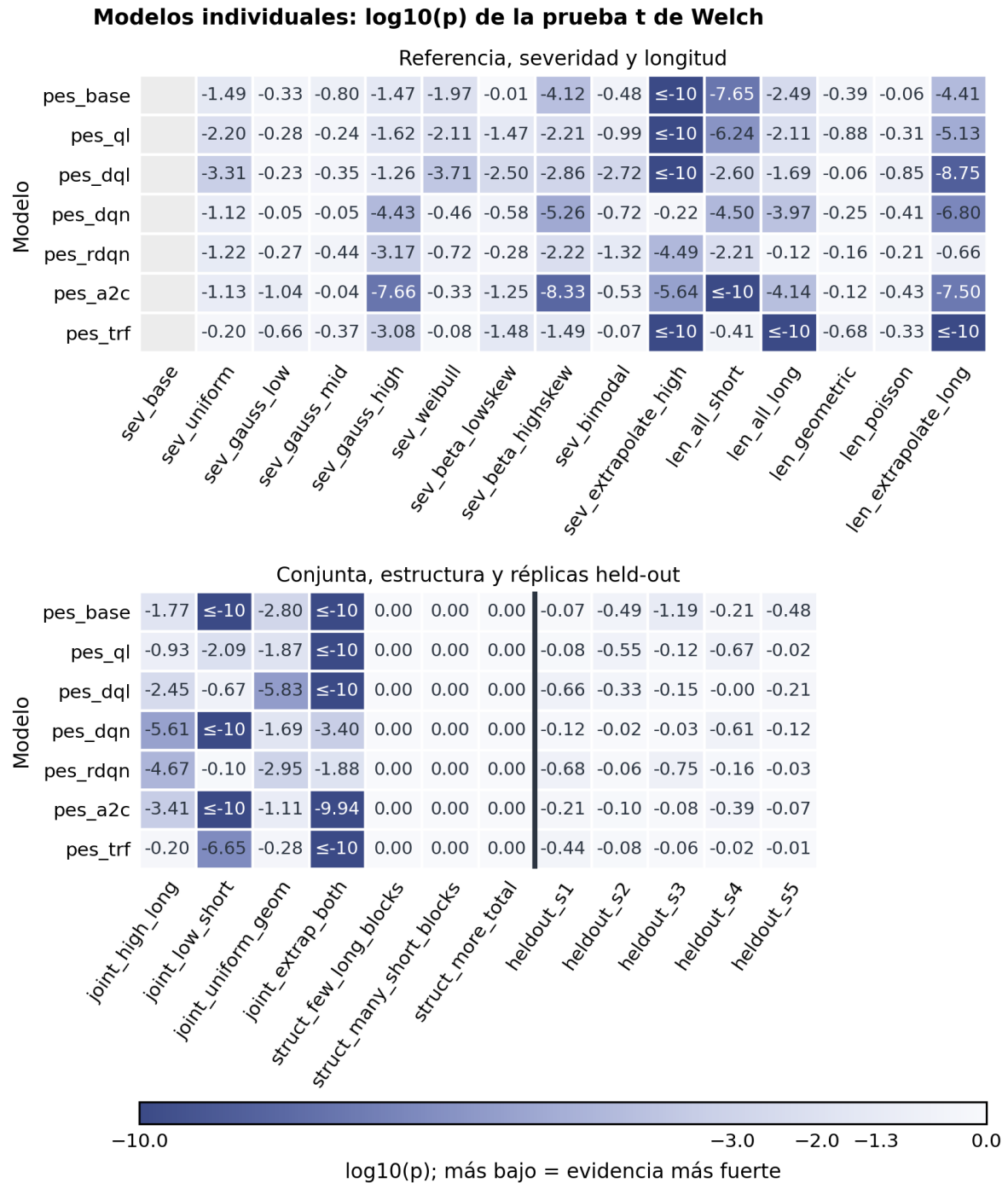
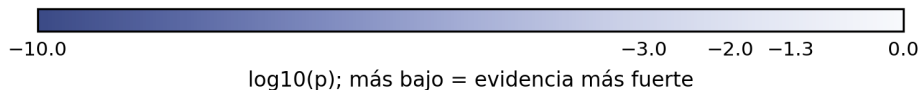


Figura C.2: Modelos individuales: $\log_{10} p$ de Welch de cada escenario frente a la referencia del mismo modelo; los valores menores que -10 se muestran como ≤ -10 . En el panel inferior, a la derecha de la línea vertical, las cinco réplicas *held-out*.



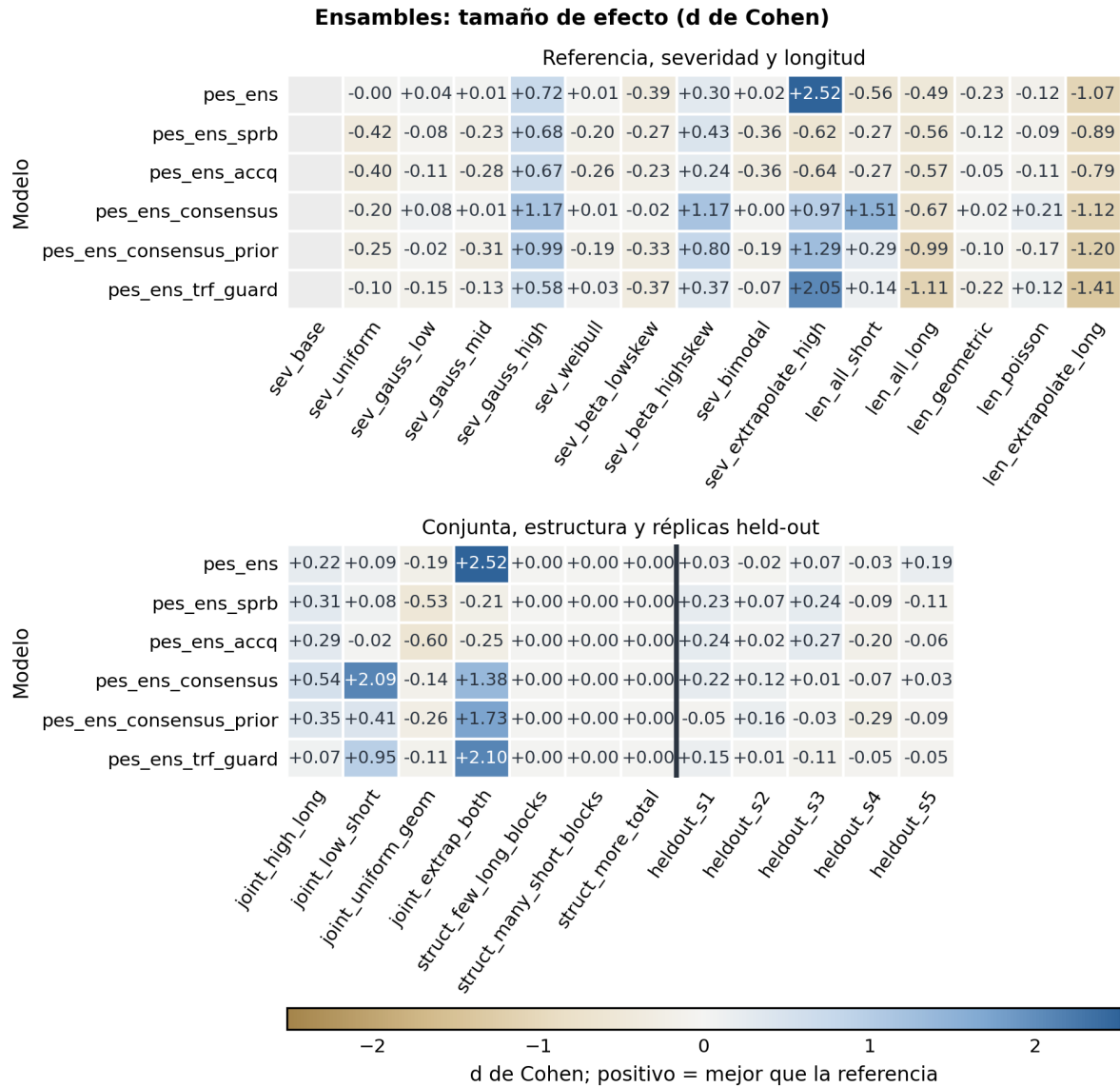


Figura C.5: Ensamblas: d de Cohen de cada escenario frente a la referencia del mismo ensamble (valores positivos: mejor que la referencia). En el panel inferior, a la derecha de la línea vertical, las cinco réplicas *held-out*.

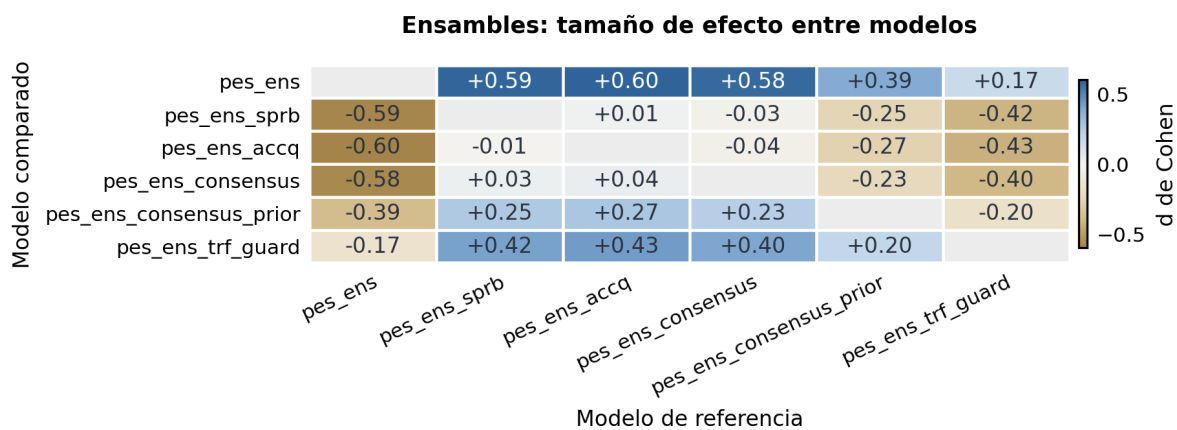


Figura C.6: Ensamblas: d de Cohen entre pares de ensambles en los 21 escenarios de generalización.

D. Resultados por secuencia de cada modelo

Las figuras de esta sección muestran, para cada modelo individual, el desempeño en cada secuencia de la referencia, su distribución y sus estadísticos por bloque (Sección 5.1).

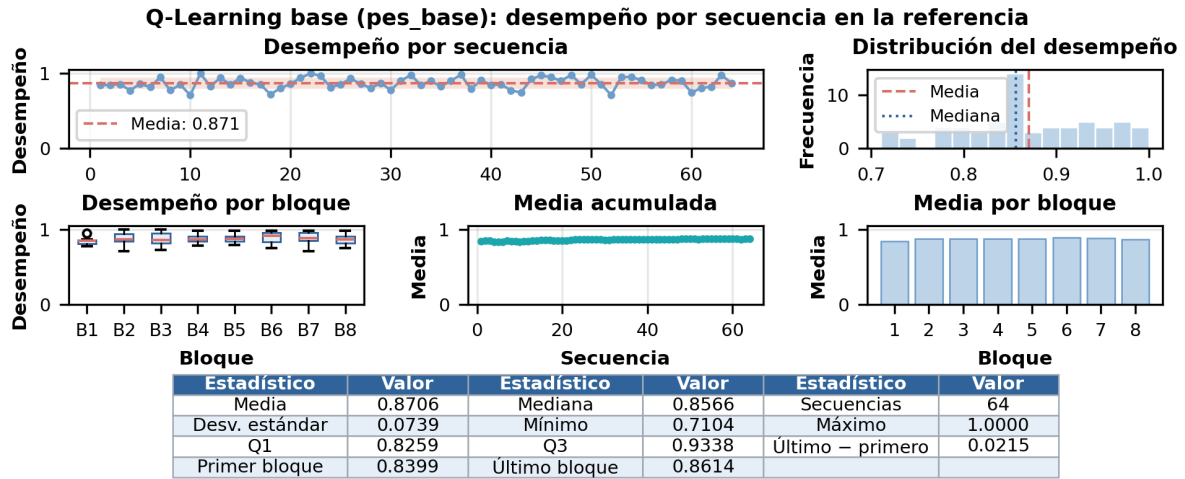


Figura D.1: Resultados por secuencia de Q-Learning base (pes_base) en la referencia.

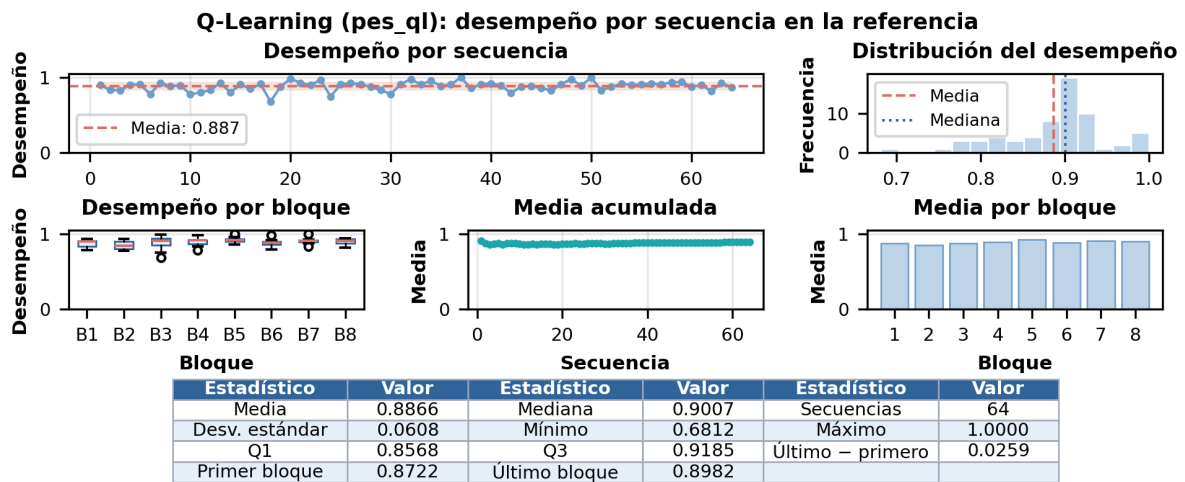


Figura D.2: Resultados por secuencia de Q-Learning (pes_q1) en la referencia.

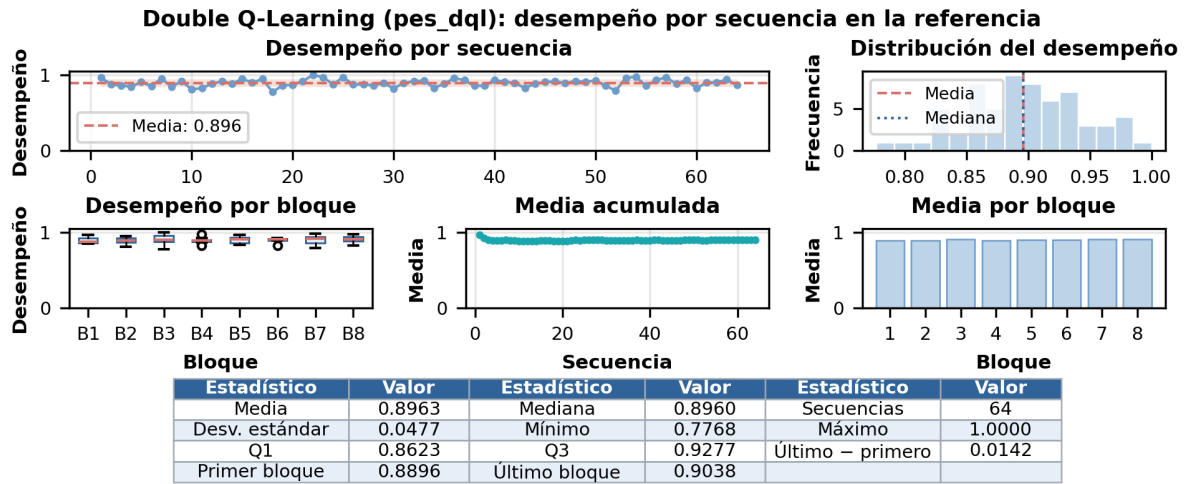


Figura D.3: Resultados por secuencia de Double Q-Learning (pes_dql) en la referencia.

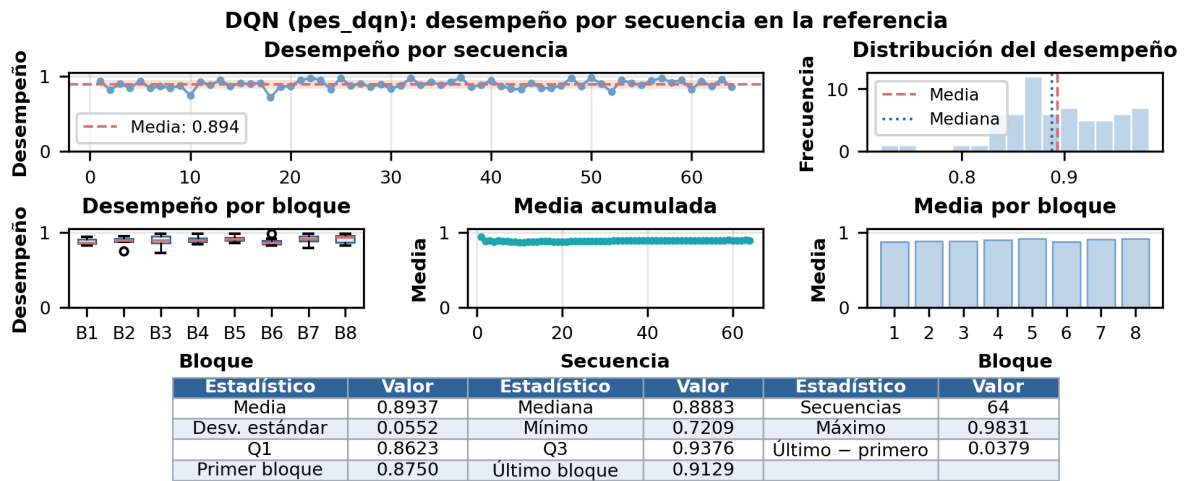


Figura D.4: Resultados por secuencia de DQN (pes_dqn) en la referencia.

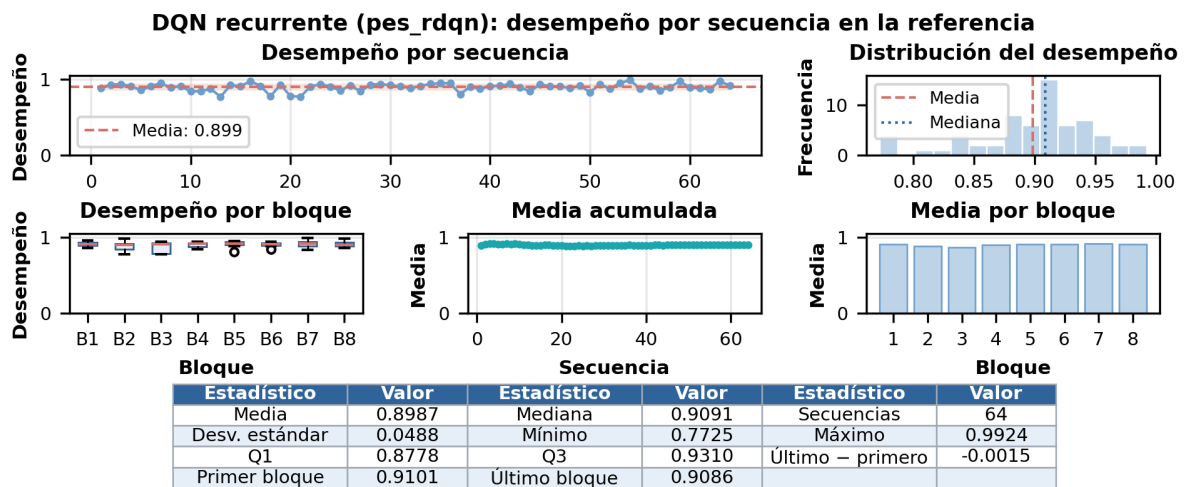


Figura D.5: Resultados por secuencia del DQN recurrente (pes_rdqn) en la referencia.

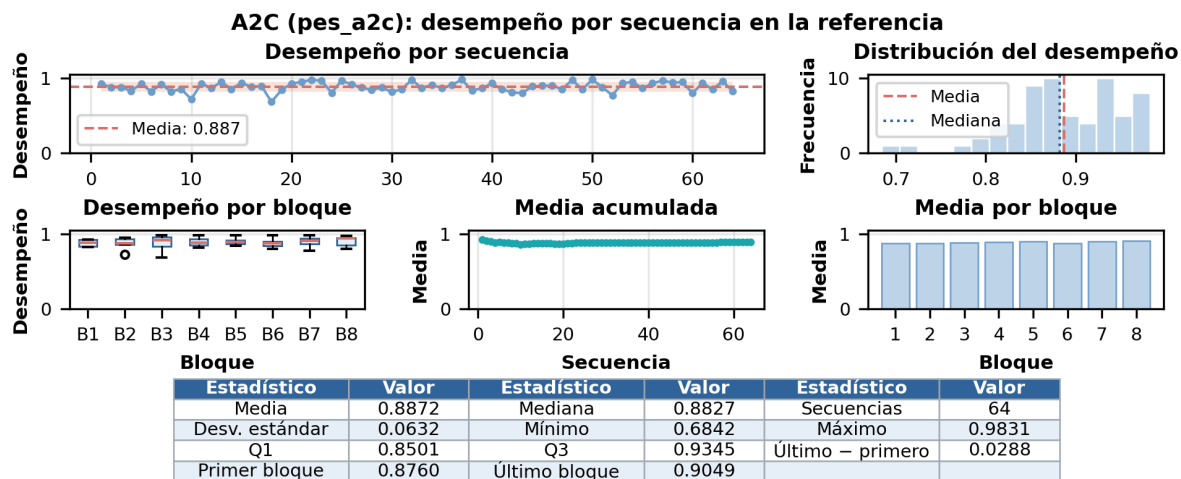


Figura D.6: Resultados por secuencia de A2C (pes_a2c) en la referencia.

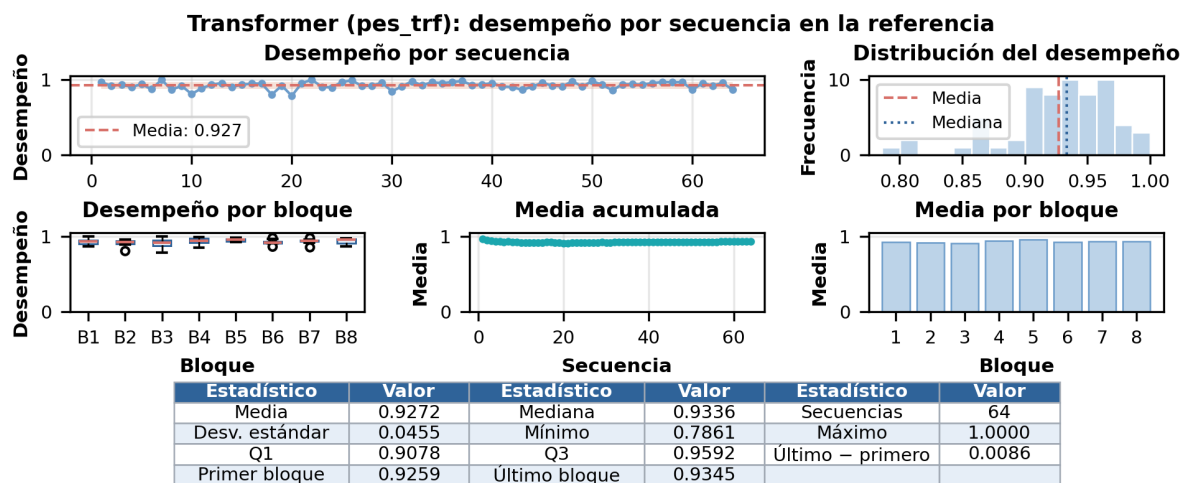


Figura D.7: Resultados por secuencia del Transformer (pes_trf) en la referencia.